

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Поволжский государственный университет телекоммуникаций и информатики»

Факультет

Кибербезопасности и управления

Направление
/специальность

подготовки

09.03.01

Информатика и вычислительная техника

Кафедра

Программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
(БАКАЛАВРСКАЯ РАБОТА)

**Разработка с использованием JavaScript веб-приложения для
изучения иностранных языков**

Утверждаю

и.о. зав. каф.

к.т.н., доцент

И. С. Макаров

Должность

Уч. степень, звание

Подпись

Дата

Инициалы Фамилия

Руководитель

ст. препод.

В. С. Ушаков

Н. контролер

ст. препод.

Е. В. Расеева

Разработал

ИВТ-22

М. А. Белоус

Группа

Подпись

Дата

Инициалы Фамилия

Самара, 2026

Содержание

Задание.....	3
Отзыв руководителя.....	5
Показатели качества ВКР.....	7
Реферат.....	8
Введение.....	9
1 Анализ предметной области и постановка задачи на разработку веб-приложения для изучения иностранных языков.....	12
1.1 Технологии интервального повторения в современных системах обучения.....	12
1.2 Обзор и анализ существующих программных решений (аналогов).....	16
1.3 Методологическая основа и обоснование выбора технологий.....	18
1.4 Постановка задачи на разработку системы.....	19
2 Проектирование информационной системы.....	21
2.1 Архитектура разрабатываемого приложения.....	21
2.2 Проектирование базы данных.....	23
2.3 Проектирование REST API.....	30
2.4 Проектирование алгоритма интервального повторения.....	34
2.5 Выбор и обоснование технологий.....	37
3 Реализация программного обеспечения.....	39
3.1 Организация серверной части (Backend).....	39
3.2 Организация клиентской части (Frontend).....	42
3.3 Интеграция с внешним API перевода.....	49
3.4 Стилизация и адаптивный дизайн.....	50
4 Тестирование и анализ результатов.....	56
4.1 Цели и задачи тестирования.....	56
4.2 Сценарии функционального тестирования.....	56
4.3 Модульное тестирование (Unit – тесты).....	62
4.4 Результат тестирования алгоритма интервального повторения.....	68
Заключение.....	69
Список оспользованных источников.....	73
Приложение А – Скриншоты интерфейса приложения.....	74
Приложение Б – Листинг программного кода.....	77

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Поволжский государственный университет телекоммуникаций и
информатики»

ЗАДАНИЕ

по подготовке выпускной квалификационной работы

Студента

Белоус Михаила Алексеевича

Тема ВКР

Разработка с использованием JavaScript
веб-приложения для изучения
иностранных языков

Утверждена приказом по университету от

16.04.2026 № 115-2

Срок сдачи студентом законченной ВКР

09.06.26

Исходный данные и постановка задачи

- 1) Технологии: JavaScript, Node.js, Express.js, React, Vite, SQLite, JWT.
- 2) Требования: регистрация, изучение слов с интервальным повторением, добавление слов, статистика.
- 3) Разработать веб-приложение для изучения иностранных слов на базе алгоритма SM-2.
- 4) Реализовать бэкенд на Express.js с SQLite и JWT-аутентификацией.
- 5) Реализовать фронтенд на React с карточками слов и кнопками оценки.
- 6) Провести тестирование и оформить пояснительную записку.

Перечень подлежащих разработке в ВКР

вопросов или краткое содержание ВКР. Сроки исполнения 20.04.2026

- 1) Анализ алгоритмов интервального повторения (SM-2, SM-4, SM-8) и существующих аналогов.
- 2) Проектирование архитектуры приложения и базы данных SQLite.
- 3) Разработка серверной части на Node.js + Express.js с JWT-аутентификацией.
- 4) Разработка клиентской части на React + Vite (карточки слов, кнопки оценки, статистики).
- 5) Реализация алгоритма инетервального повторения (уровни 0-5, интервалы [0, 1, 3, 7, 14, 30])
- 6) Интеграция с API MyMemory для автоматического перевода.
- 7) Тестирование (функциональное и модульное), анализ результатов.
- 8) Оформление пояснительной записки в соответствии с ГОСТ.

Перечень графического материала.

Сроки исполнения 28.05.2026

Презентация в MS PowerPoint

Дата выдачи задания « 17 » апреля 2026 г.

кафедра

Программной инженерии

утверждаю

и.о. зав.кафедрой к.т.н., доцент

И.С. Макаров

Должность

Уч.степень, звание

Подпись

Дата

Инициалы Фамилия

руководитель

ст.препод.

В.С. Ушаков

Должность

Уч.степень, звание

Подпись

Дата

Инициалы Фамилия

Задание принял

к исполнению

ИВТ-22

М. А. Белоус

Группа

Подпись

Дата

Инициалы Фамилия

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
**Федеральное государственное бюджетное образовательное учреждение
высшего образования**
«Поволжский государственный университет телекоммуникаций и информатики»

ОТЗЫВ РУКОВОДИТЕЛЯ

Тип ВКР	<i>Бакалаврская работа</i>
Студента(ки)	<i>Белоус Михаила Алексеевича</i>
Направление подготовки/ специальность	<i>Информатика и вычислительная техника</i>
Тема ВКР	<i>Разработка с использованием JavaScript веб-приложения для изучения иностранных языков</i>
Руководитель	<i>Виктор Сергеевич Ушаков</i>
Ученая степень, звание	
Место работы (должность)	<i>Старший преподаватель кафедры программной инженерии</i>

АКТУАЛЬНОСТЬ ТЕМЫ

Тема ВКР является актуальной, поскольку самостоятельное изучение иностранных языков требует эффективных инструментов для запоминания лексики. Существующие аналоги (Anki, Quizlet, Duolingo) либо сложны для начинающих пользователей, либо имеют платные ограничения. Разработанное веб-приложение на основе алгоритма интервального повторения SM-2 решает эту проблему, предоставляя бесплатный, интуитивно понятный инструмент с автоматическим переводом слов, адаптивным дизайном и открытым исходным кодом.

ОЦЕНКА СОДЕРЖАНИЯ РАБОТЫ

(Структура, логика и стиль изложения представленного материала. глубина и степень проработки материала, обоснованность изложенных выводов, использование математического аппарата, использование средств вычислительной техники, макетирование, моделирование, экспериментирование)

Работа имеет логичную структуру, материал изложен последовательно и аргументированно. Проведён глубокий анализ аналогов и алгоритмов, детально проработана архитектура и база данных. Выводы обоснованы и подтверждены 33 автоматическими тестами. Использован математический аппарат (формулы SM-2). Разработка велась в современных средах, выполнены ER-диаграмма, блок-схема, функциональное и модульное тестирование.

СТЕПЕНЬ ДОСТИЖЕНИЯ ЦЕЛИ И ПРАКТИЧЕСКАЯ ЗНАЧИМОСТЬ

(Полнота раскрытия исследуемой темы, практическая ценность и возможность внедрения)

Поставленная цель — разработка полнофункционального веб-приложения для изучения иностранных языков с системой интервального повторения — достигнута в полном объёме. Белоус Михаилом Алексеевичем решены все задачи: проведён анализ аналогов и алгоритмов, спроектирована архитектура и база данных, реализованы серверная и клиентская части, внедрён алгоритм SM-2 с уровнями 0–5 и интервалами [0,1,3,7,14,30] дней, создан стартовый набор из 30 слов, выполнено тестирование (33 автоматических теста успешно пройдены). Практическая значимость работы заключается в том, что разработанное приложение может быть использовано для самостоятельного изучения иностранной лексики. Ключевые преимущества: автоматический перевод слов через MyMemory API, тёмная тема, адаптивный дизайн для мобильных устройств, открытый исходный код на GitHub. Приложение готово к развёртыванию и использованию.

ЗАКЛЮЧЕНИЯ ПО ПРЕДСТАВЛЕННОЙ РАБОТЕ

(Степень самостоятельной работы студента; совокупная оценка труда студента и его квалификация)

Студент Белоус Михаил Алексеевич выполнил работу самостоятельно, на всех этапах проявляя инициативу и ответственность. Код написан с нуля, тесты разработаны самостоятельно. Продемонстрировано владение современными технологиями веб-разработки. Бакалаврская работа соответствует требованиям, предъявляемым к ВКР, заслуживает оценки «отлично», а студенту Белоус Михаилу Алексеевичу может быть присвоена квалификация «бакалавр» по направлению подготовки 09.03.01 «Информатика и вычислительная техника».

Руководитель ВКР

Подпись

Дата

В.С. Ушаков

Инициалы Фамилия

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Поволжский государственный университет телекоммуникаций и
информатики»

ПОКАЗАТЕЛИ КАЧЕСТВА ВКР

На тему

Разработка с использованием JavaScript веб-приложения для
изучения иностранных языков

1 Работа выполнена:

- по теме, предложенной студентом
- по заявке предприятия
- в области фундаментальных и поисковых научных исследований

☐☐☐

наименование предприятия

указать область исследований

2 Результаты ВКР:

- рекомендованы к опубликованию
- рекомендованы к внедрению
- внедрены

☐☐☐

указать где

указать где

акт внедрения

3 ВКР имеет практическую ценность

☐

Разработка с использованием JavaScript веб-приложения для изучения иностранных языков

☐

в чём заключается практическая ценность

4 Использование ЭВМ при выполнении ВКР: (ПО, компьютерное моделирование, компьютерная обработка данных и др.)

VS Code, Git, Node.js, Vite, Vitest, SQLite, Postman, Chrome DevTools.

Наличие электронной версии с

% оригинальности

Должность

Подпись

Дата

Инициалы Фамилия

Студент

ИВТ-22

Белоус М. А.

Группа

Подпись

Дата

Инициалы Фамилия

Руководитель
ВКР

ст.препод.

Ушаков В. С.

Должность

Уч. степень

Подпись

Дата

Инициалы Фамилия

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И
МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Поволжский государственный университет телекоммуникаций и
информатики»

РЕФЕРАТ

Название	«Разработка с использованием JavaScript веб-приложения для изучения иностранных языков»
Автор	Белоус Михаил Алексеевич
Руководитель ВКР	Ушаков Виктор Сергеевич
Ключевые слова	интервальное повторение, алгоритм SM-2, React, Express.js, SQLite, JWT-аутентификация, флеш-карточки, JavaScript.
Дата публикации	2026
Библиографическое описание	Белоус, М. А. Разработка с использованием JavaScript веб-приложения для изучения иностранных языков[Текст]: бакалаврская работа / М. А. Белоус; Поволжский государственный университет телекоммуникаций и информатики (ПГУТИ). Кафедра программной инженерии (При); рук. ВКР – В. С. Ушаков. – Самара, 2026. – 79 с.
Аннотация	Разработано веб-приложение для изучения иностранных языков с системой интервального повторения на основе алгоритма SM-2. Приложение использует React, Express.js, SQLite и JWT-аутентификацию. Обеспечивает добавление и изучение слов, автоматический перевод, отслеживание статистики. Тестирование подтвердило работоспособность всех модулей.

Руководитель ВКР

Подпись

Дата

Ушаков В. С.
Инициалы Фамилия

Введение

В современном мире знание иностранных языков становится не просто преимуществом, а необходимым навыком для профессионального и личностного роста. Однако традиционные методы обучения (учебники, курсы) часто не учитывают индивидуальные особенности запоминания слов, что приводит к низкой эффективности и быстрому забыванию материала. Особенно остро эта проблема стоит при самостоятельном изучении языка, где отсутствует внешний контроль и система повторения.

Цифровые технологии предлагают решение в виде мобильных и веб-приложений, использующих алгоритмы интервального повторения. Такие системы, как Anki или Quizlet, показывают высокую эффективность, но они либо имеют ограничения в бесплатной версии, либо перегружены функциями, не всегда необходимыми начинающим пользователям. Кроме того, многие из них не позволяют гибко настраивать процесс обучения под личные потребности.

Все вышесказанное определило актуальность темы работы – разработка с использованием JavaScript веб-приложения для изучения иностранных языков, основанного на научно обоснованном алгоритме интервального повторения SM-2, с интуитивно понятным интерфейсом и полным контролем над процессом обучения.

Целью ВКР является разработка и реализация полнофункционального веб-приложения для изучения иностранных языков с системой интервального повторения, обеспечивающего эффективное запоминание лексики и отслеживание прогресса пользователя.

Для достижения поставленной цели необходимо решить следующие основные задачи:

1. Провести анализ существующих аналогов и алгоритмов интервального повторения (SM-2, SM-4, SM-8).

2. Сформулировать требования к разрабатываемой системе на основе выявленных недостатков существующих решений.
3. Спроектировать архитектуру приложения, включая базу данных, серверную часть и клиентский интерфейс.
4. Обосновать выбор технологического стека (React + Vite, Express.js, SQLite, JWT).
5. Разработать схему API-эндпоинтов и блок-схему алгоритма интервального повторения.
6. Реализовать backend-часть на Express.js с JWT-аутентификацией и SQLite-базой данных.
7. Разработать frontend-часть на React + Vite, включающую страницы аутентификации, карточек слов, статистики и профиля.
8. Внедрить алгоритм интервального повторения для автоматического планирования показов слов.
9. Создать стартовый набор из 30 учебных слов по 5 тематическим категориям.
10. Провести тестирование работоспособности, оценить эффективность системы и сравнить её с аналогами.
11. Определить направление для дальнейшего развития приложения.

Объектом исследования выступает процесс самостоятельного изучения иностранной лексики с использованием цифровых технологий.

Предметом исследования является методы и алгоритмы интервального повторения (в частности, SM-2) и их программная реализация в виде веб-приложения.

Основными источниками информации для написания работы послужили научные публикации по алгоритмам интервального повторения (Пётр Возняк), документация по используемым технологиям (React, Express, SQLite, JWT), анализ существующих приложений (Anki, Quizlet, Duolingo), а также материалы открытых онлайн-курсов по веб-разработке.

Цель и задачи написания работы определили её структуру, которая состоит из введения, четырёх глав и заключения.

Во введении обосновывается актуальность работы, цель, задачи, объект и предмет исследования.

Первая глава посвящена аналитическому обзору: рассматриваются существующие аналоги, анализируются алгоритмы интервального повторения (SM-2, SM-4, SM-8), выявляются их достоинства и недостатки, формируются требования к разрабатываемому приложению.

Вторая глава описывает проектирование системы: функциональная структура, ER – диаграмма базы данных SQLite, схема API-эндпоинтов, блок-схема алгоритма интервального повторения, а также обоснование выбора технологий (React + Vite, Express.js, JWT).

Третья глава содержит детальное описание реализации программного обеспечения: разработка backend-части (маршруты, JWT-аутентификация, middleware, работа с SQLite), создание frontend-компонентов (авторизации, карточки слов, статистика, профиль), интеграция алгоритма интервального повторения и наполнение базы стартовыми словами.

Четвёртая глава посвящена тестированию и анализу результатов: разрабатываются сценарии тестирования, проверяется работоспособность всех модулей, анализируется эффективность алгоритма интервального повторения, проводится сравнение с аналогами по ключевым критериям, а также предлагаются направления для дальнейшего развития приложения (мобильная версия, поддержка звука, облачное хранение).

В заключении представлены основные выводы и результаты по проделанной работе, подтверждающие достижение поставленной цели и решение всех задач.

1 Анализ предметной области и постановка задачи на разработку веб-приложения для изучения иностранных языков

1.1 Технологии интервального повторения в современных системах обучения

В современном мире, характеризующемся высокими темпами роста информационных потоков и глобализацией, владение иностранными языками становится критически важным для профессиональной деятельности специалиста в области информационных технологий и телекоммуникаций. Традиционные методы изучения лексики, основанные на одноразовом заучивании, демонстрируют низкую эффективность. Согласно кривой забывания Германа Эббингауза, человек теряет до 80% новой информации в течение первых 48 часов при отсутствии повторения.

Цифровая трансформация образования привела к появлению *систем интервального повторения (Spaced Repetition Systems, SRS)*, которые алгоритмически рассчитывают оптимальные моменты для повторения материала, минимизируя время забывания.

В истории развития технологий запоминания ключевую роль сыграл Пётр Возняк, создавший семейство алгоритмов SuperMemo (SM). Для реализации дипломного проекта представляет интерес сравнение трёх поколений этих алгоритмов, представленное в таблице 1.1.

Таблица 1.1

Сравнительная характеристика алгоритмов семейства SM

Параметр / Алгоритм	SM-2 (1987)	SM-4 (1989)	SM-8 (1999)
Базовый принцип	Фиксированные множители	Адаптация множителей	Теория информации
Фактор лёгкости (EF)	Модифицируется линейно	Модифицируется сложнее	Многомерная матрица
Реакция на ошибку	Сброс интервала + понижение EF	Частичный сброс интервала	Плавное изменение без полного сброса

Вычислительная сложность	Низкая ($O(1)$)	Средняя	Высокая (Big Data)
--------------------------	-------------------	---------	--------------------

Анализ таблицы:

- SM-8, несмотря на высокую теоретическую эффективность, требует накопления обширной статистики ответов пользователя для построения многомерной матрицы повторений. Это приводит к «холодному старту»: в начальный период обучения (первые 100–200 карточек) алгоритм показывает случайные или неоптимальные интервалы. Учитывая ограниченный горизонт дипломного проектирования и необходимость демонстрации немедленной работоспособности, внедрение SM-8 является избыточным.

- SM-2, напротив, начинает работать с первого дня, используя фиксированные множители, и показывает линейную сходимость к оптимальному графику повторений уже на малых выборках данных (30–50 слов).

Таким образом, алгоритм SM-2 выбран в качестве основы для реализации, так как он обеспечивает детерминированный результат «здесь и сейчас» и имеет вычислительную сложность $O(1)$, что критически важно для веб-приложений с реальным режимом времени.

В разработанном приложении используется упрощённая реализация алгоритма интервального повторения, основанная на принципах семейства SM, но адаптированная для простоты кодирования и наглядности для пользователя.

Уровни сложности слов

Каждое слово пользователя находится на одном из 6 уровней (level от 0 до 5). Каждому уровню соответствует фиксированный интервал до следующего повторения, представленный в таблице 1.2.

Таблица 1.2

Интервалы повторения в зависимости от уровня слова

Уровень	Интервал (дней)	Пояснение
0	0	Новое слово – немедленное повторение
1	1	Повтор на следующий день
2	3	Повтор через 3 дня
3	7	Повтор через неделю
4	14	Повтор через две недели
5	30	Повтор через месяц

Выбор интервалов основан на практических рекомендациях для систем интервального повторения: начальные интервалы короткие (1–3 дня), затем постепенно увеличиваются до недели, двух недель и месяца. Максимальный интервал в 30 дней выбран как разумный компромисс между эффективностью запоминания и комфортом пользователя.

Шкала оценок пользователя

При повторении слова пользователь выбирает одну из четырёх кнопок оценки. Каждой кнопке соответствует числовое значение *quality*, которое передаётся на сервер для расчёта нового уровня слова. Соответствие представлено в таблице 1.3.

Таблица 1.3

Маппинг пользовательских оценок

Кнопка	<i>quality</i>	Смысл
Сложно	0	Полная ошибка, слово не запомнено
Не очень	2	Вспомнил с трудом, требуется повторение
Хорошо	4	Нормально, уверенное знание
Отлично	5	Идеально, лёгкое

		запоминание
--	--	-------------

Обратите внимание, что значения $quality = 1$ и $quality = 3$ не используются в приложении. Это упрощение было принято для улучшения пользовательского опыта: четыре кнопки легче воспринимаются и быстрее нажимаются, чем пять или шесть. При этом выбранные значения (0, 2, 4, 5) позволяют корректно работать логике пересчёта уровней.

Логика изменения уровня слова

На основе полученного значения $quality$ уровень слова пересчитывается по следующим правилам:

1. Если $quality \geq 4$ (оценки «Хорошо» или «Отлично») — уровень повышается на 1, но не более максимального (5)
2. Если $quality \geq 2$ (оценка «Не очень») — уровень остаётся неизменным
3. Если $quality < 2$ (оценка «Сложно») — уровень понижается на 1, но не менее минимального (0)

На псевдокоде эту логику можно выразить следующим образом:

```
if quality >= 4:
    new_level = min(5, current_level + 1)
else if quality >= 2:
    new_level = current_level
else:
    new_level = max(0, current_level - 1)
```

После обновления уровня вычисляется новая дата следующего повторения по формуле:

$next_review = \text{текущая_дата} + \text{интервал}[\text{новый_уровень}]$,

где $\text{интервал}[\text{новый_уровень}]$ – значение из таблицы 1.2.

Пример работы алгоритма

Пусть пользователь изучает слово, находящееся на уровне 2 (интервал 3 дня). При разных оценках произойдут следующие изменения, представленные в таблице 1.4.

Таблица 1.4

Пример работы алгоритма

Текущий уровень	Оценка пользователя	quality	Новый уровень	Новый интервал
2	Отлично	5	3	7 дней
2	Хорошо	4	3	7 дней
2	Не очень	2	2	3 дня
2	Сложно	0	1	1 день

Из таблицы видно, что:

- При положительных оценках (4–5) слово прогрессирует к более высокому уровню, увеличивая интервал между повторениями
- При нейтральной оценке (2) слово остаётся на текущем уровне, интервал не меняется
- При отрицательной оценке (0) слово опускается на уровень ниже, интервал сокращается, что заставляет пользователя повторить слово раньше

1.2 Обзор и анализ существующих программных решений (аналогов)

Анализ рынка проводился по трём наиболее популярным продуктам, представленным в таблице 1.5.

Таблица 1.5

Сравнительный анализ аналогов

Критерий	Anki	Quizlet	Duolingo	Разрабатываемое приложение
Тип алгоритма SRS	Модифицированный SM-2	Проприетарный (Premium)	Проприетарный	Упрощённый SM-2
Лицензия	Freemium (плата за iOS)	Freemium (жёсткие лимиты)	Freemium (реклама)	Полностью бесплатный
Шкала оценок	4 кнопки	4 кнопки	3 кнопки	4 кнопки
Автоматический перевод	Нет (только ручной ввод)	Нет	Есть (встроенный словарь)	Есть (MyMemory API)
Веб-интерфейс	Устаревший, высокий порог входа	Современный	Игровой, отвлекающий	Минималистичный (React)
Возможность добавления слов	Да (сложно)	Да (просто)	Ограничена	Да (просто)
Отслеживание статистики	Подробная	Базовая	Игровая	Подробная (по уровням и категориям)

Выводы по анализу аналогов:

Существующие решения обладают следующими недостатками:

1. Anki — сложен для неопытного пользователя, требует ручной настройки, имеет устаревший интерфейс.
2. Quizlet — ключевая функция долгосрочного запоминания находится за платным доступом (Premium).
3. Duolingo — геймификация часто мешает серьёзному запоминанию, алгоритм непрозрачен и не даёт пользователю полного контроля над процессом обучения.

Следовательно, создание собственного легковесного веб-приложения с открытым и понятным алгоритмом интервального повторения является актуальной задачей. При этом разработанное приложение использует интуитивно понятную шкалу из 4 уровней оценки, что сопоставимо с популярными аналогами (Anki) и упрощает взаимодействие пользователя с системой.

1.3 Методологическая основа и обоснование выбора технологий

Для реализации поставленной цели была выбрана архитектура Client-Server (REST API). Такой подход обеспечивает чёткое разделение ответственности: сервер отвечает за бизнес-логику (алгоритм интервального повторения) и хранение данных, а клиент — за отображение интерфейса и взаимодействие с пользователем.

Таблица 1.6

Обоснование выбора технологического стека

Компонент	Технология	Обоснование (соответствие задачам)
Сервер (Backend)	Node.js + Express.js	Единый язык с клиентом (JavaScript), высокая производительность ввода-вывода, огромная экосистема модулей, простота создания REST API.
База данных	SQLite	Не требует отдельного сервера, идеально для дипломного проекта (переносимость одного файла БД), поддерживает сложные SQL-запросы для выборки слов, ACID-совместимость.

Клиент (Frontend)	React + Vite	Декларативный подход к созданию пользовательского интерфейса, эффективное управление состоянием карточек, быстрая «горячая» перезагрузка при разработке (Vite).
-------------------	--------------	---

Продолжение табл.1.6

Аутентификация	JWT (JSON Web Token)	Стандарт для SPA (Single Page Applications), отсутствие состояния на сервере (не нужно хранить сессии), безопасная передача данных пользователя.
Хеширование паролей	bcrypt	Надёжный алгоритм хеширования с «солью», устойчивый к атакам перебором.

1.4 Постановка задачи на разработку системы

На основе проведённого анализа была сформулирована задача на создание веб-приложения «Интерактивный тренажёр лексики», отвечающего следующим требованиям (табл. 1.7, 1.8).

Таблица 1.7

Функциональные требования

ID	Требование	Описание
FR01	Аутентификация	Обеспечить регистрацию и вход пользователей (email + password) с хешированием пароля и выдачей JWT-токена (срок действия 30 дней).
FR02	Управление словарём	Реализовать возможность добавления новых слов (с переводом, примером использования и категорией) как вручную, так и с использованием автоматического перевода.

FR03	Алгоритм интервального повторения	Внедрить ядро системы — алгоритм пересчёта уровня слова и даты следующего показа на основе 4-уровневой шкалы оценок (0, 2, 4, 5).
------	-----------------------------------	---

Продолжение табл. 1.7

FR04	Режим изучения (тренажёр)	Организовать выборку слов, у которых дата следующего повтора (next_review) меньше или равна текущей дате. Отображать карточки с вопросом и ответом.
FR05	Статистика	Собирать и отображать метаданные о процессе обучения: общее количество слов, изученные слова (уровень ≥ 3), слова к повторению сегодня, разбивка по уровням и категориям.
FR06	Стартовый контент	Создать и наполнить базу данных 30 стартовыми словами по 5 темам: «Основы», «Еда», «Путешествия», «Работа», «Хобби».
FR07	Автоматический перевод	При добавлении нового слова реализовать возможность автоматического получения перевода с использованием внешнего API (MyMemory Translate) для упрощения заполнения карточек.

Таблица 1.8

Нефункциональные требования

ID	Требование	Описание
NFR01	Производительность	Время отклика API на запрос получения следующей карточки не должно превышать 300 мс при стандартной нагрузке.
NFR02	Кросс-платформенность	Приложение должно корректно отображаться в актуальных версиях браузеров (Chrome, Firefox, Safari, Edge).

NFR03	Безопасность	Пароли должны храниться в хешированном виде (bcrypt). Доступ к API должен осуществляться только по валидному JWT-токену.
NFR04	Надёжность	Данные пользователя не должны теряться при сбоях. База данных SQLite должна поддерживать целостность внешних ключей.
NFR05	Читаемость кода	Исходный код должен быть написан с соблюдением соглашений о стиле, обеспечивающих читаемость и поддерживаемость.

2 Проектирование информационных системы

2.1 Архитектура разрабатываемого приложения

Разрабатываемое веб-приложение «Интерактивный тренажёр лексики» построено на основе клиент-серверной архитектуры с использованием REST API. Такой подход обеспечивает чёткое разделение ответственности между компонентами системы и позволяет независимо развивать клиентскую и серверную части.

На рис. 2.1 представлена общая архитектура системы.

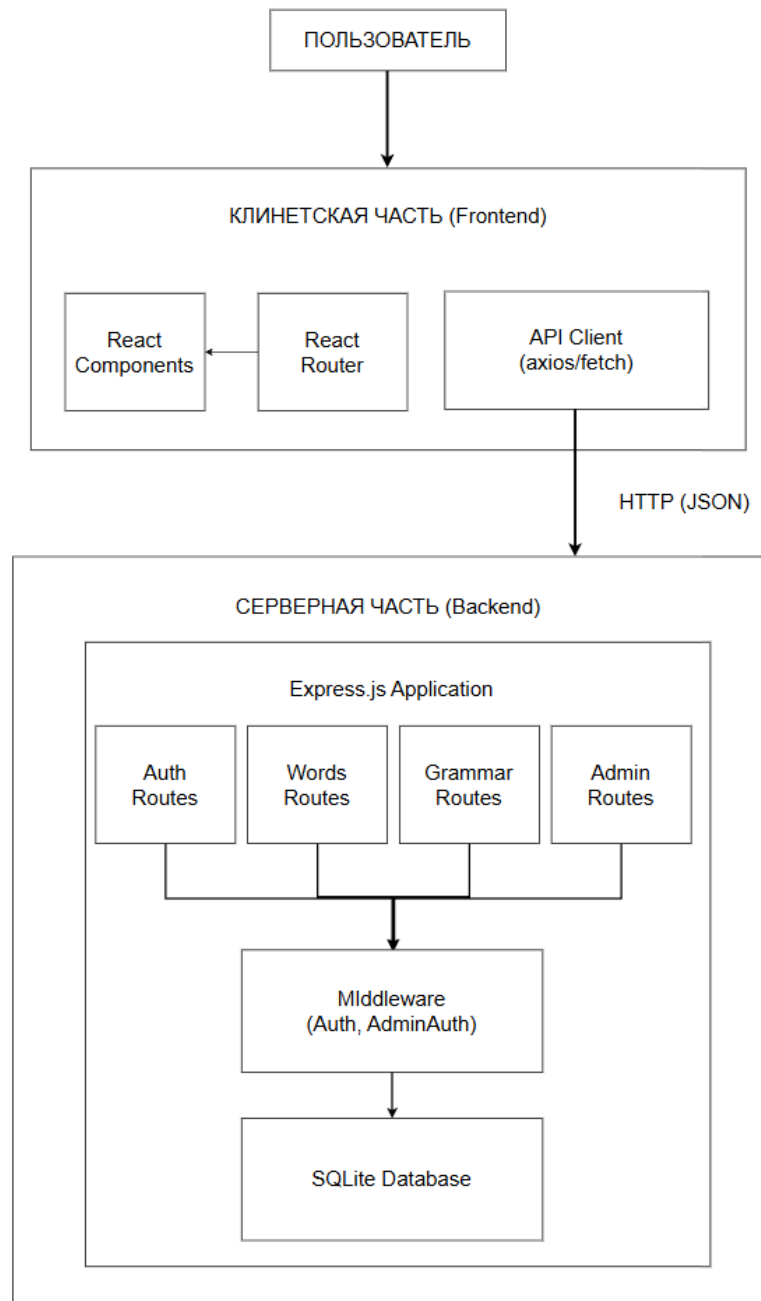


Рис. 2.1 – Архитектура веб-приложения

Клиентская часть (Frontend):

- Реализована на React с использованием Vite в качестве сборщика
- Отвечает за отображение пользовательского интерфейса и обработку действий пользователя
- Взаимодействует с сервером через API Client, который инкапсулирует все HTTP-запросы
- Сохраняет JWT-токен в localStorage для поддержания сессии

Серверная часть (Backend):

- Реализована на Node.js с использованием фреймворка Express.js
- Обеспечивает бизнес-логику приложения: аутентификацию, алгоритм интервального повторения, статистику
- Предоставляет REST API для клиента
- Использует SQLite в качестве встроенной базы данных

База данных:

- Файл базы данных app.db хранится на сервере
- Взаимодействие осуществляется через нативный sqlite3 модуль Node.js

2.2 Проектирование базы данных

На основе анализа предметной области была разработана структура базы данных (рис. 2.2), включающая следующие основные сущности:

1. Пользователь (users) — хранит информацию о зарегистрированных пользователях (таблица 2.1)
2. Слово (words) — хранит словарные карточки, принадлежащие пользователям (таблица 2.2)
3. Правило грамматики (grammar_rules) — хранит справочную информацию по грамматике (таблица 2.3)

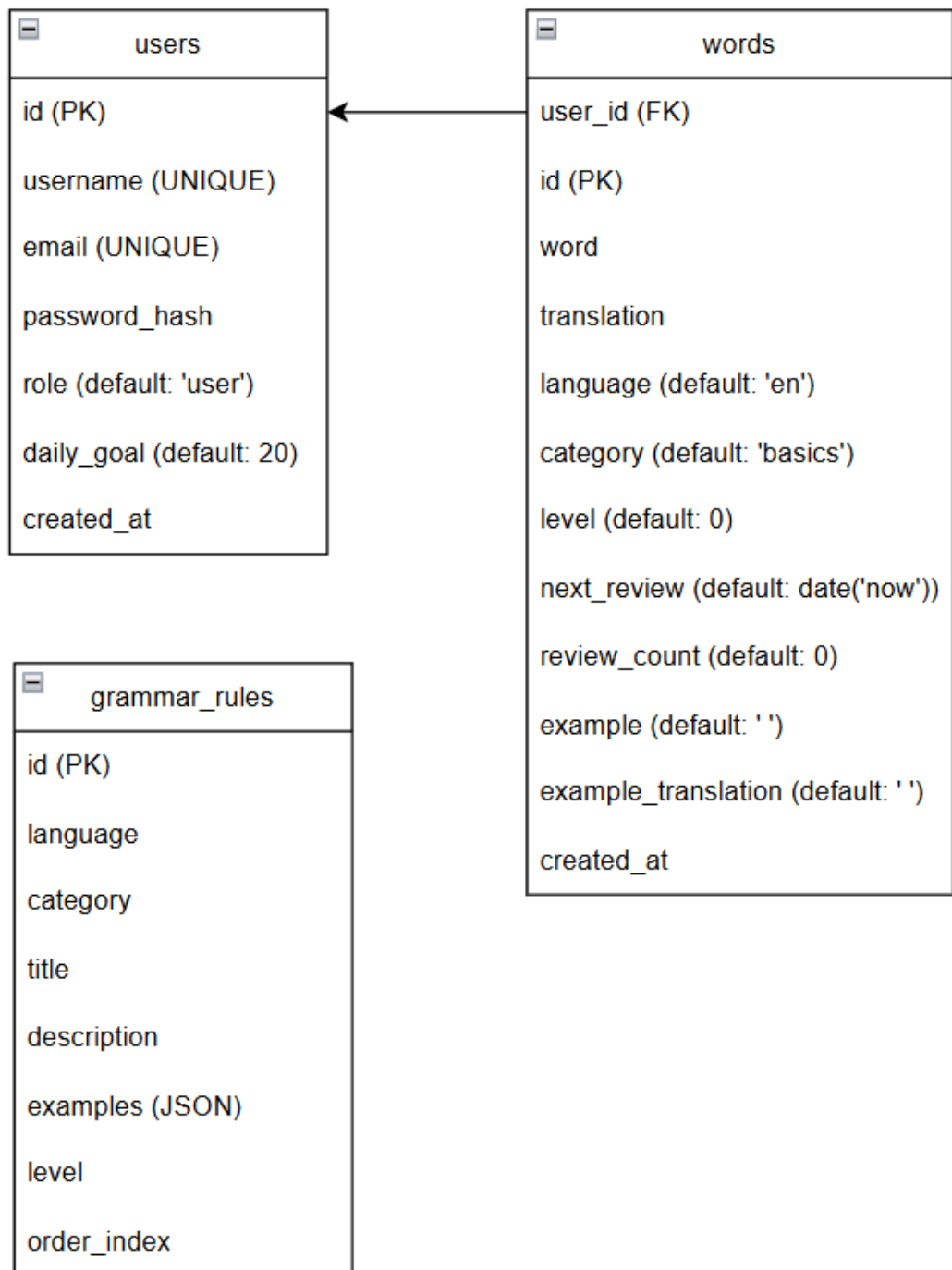


Рис. 2.2 ER-диаграмма базы данных

Особенности организации базы данных:

Инициализация базы данных разделена на два модуля:

- `server/config/database.js` — создаёт основные таблицы (`users`, `words`) и настраивает параметры производительности
- `server/grammar-seed.js` — создаёт таблицу `grammar_rules` и заполняет её стартовыми правилами

Такое разделение позволяет гибко управлять функциональными модулями приложения: при необходимости грамматику можно отключить, не затрагивая ядро системы. Таблица `grammar_rules` является справочной и не связана напрямую с таблицей `users`, поскольку грамматические правила являются общими для всех пользователей и не требуют персонализации.

Таблица .2.1

Структура таблицы 'users'

Поле	Тип	Ограничения	Описание
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Уникальный идентификатор пользователя
username	TEXT	NOT NULL, UNIQUE	Имя пользователя (логин)
email	TEXT	NOT NULL, UNIQUE	Адрес электронной почты
password_hash	TEXT	NOT NULL	Хеш пароля (bcrypt)
role	TEXT	DEFAULT 'user'	Роль пользователя (user / admin)
daily_goal	INTEGER	DEFAULT 20	Целевое количество слов в день
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Дата регистрации

Таблица 2.2

Структура таблицы 'words'

Поле	Тип	Ограничения	Описание
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Уникальный идентификатор слова
user_id	INTEGER	FOREIGN KEY REFERENCES users(id) ON DELETE CASCADE	Владелец слова
word	TEXT	NOT NULL	Слово на иностранном языке
translation	TEXT	NOT NULL	Перевод на родной язык
language	TEXT	DEFAULT 'en'	Язык изучения
category	TEXT	DEFAULT 'basics'	Тематическая категория
level	INTEGER	DEFAULT 0	Уровень изученности (0–5)
next_review	DATETIME	DEFAULT (date('now'))	Дата следующего повторения
review_count	INTEGER	DEFAULT 0	Количество повторений
example	TEXT	DEFAULT ''	Пример использования
example_translation	TEXT	DEFAULT ''	Перевод примера
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Дата добавления слова

Структура таблицы 'grammar_rules'

Поле	Тип	Ограничения	Описание
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Уникальный идентификатор правила
language	TEXT	NOT NULL	Язык (en, ru, и т.д.)
category	TEXT	NOT NULL	Категория (verbs, nouns, tenses)
title	TEXT	NOT NULL	Название правила
description	TEXT	NOT NULL	Описание правила
examples	TEXT		Примеры в формате JSON
level	INTEGER	DEFAULT 1	Уровень сложности (1–3)
order_index	INTEGER	DEFAULT 0	Порядок отображения

Для обеспечения быстрого выполнения наиболее частого запроса — получения слов, требующих повторения, — в базе данных создан составной индекс. Создание индекса выполняется при инициализации базы данных (листинг 2.1)

Листинг 2.1 – Создание индекса (server/config/database.js, фрагмент).

```
// Создание индекса для оптимизации запросов повторения
db.exec(`
  CREATE INDEX IF NOT EXISTS idx_words_review
    ON words(user_id, language, next_review);
`);
```

Остальная логика инициализации базы данных:

- Путь к файлу БД: server/data/app.db (создаётся автоматически при отсутствии)
- Используется библиотека better-sqlite3 для синхронного доступа

- Создаются таблицы `user` и `words` с полями, описанными в таблицах 2.1 - 2.2
- Предусмотрены миграции для добавления полей `role` и `example_translation` в существующих базах.

Обоснование выбора индекса:

Наиболее частый запрос в приложении — получение слов, требующих повторения. Составной индекс (`user_id`, `language`, `next_review`) оптимизирует этот запрос, так как позволяет быстро отфильтровать слова по пользователю и языку, затем эффективно применить условие по дате следующего повторения.

В таблице 2.4 описаны дополнительные настройки производительности

Таблица 2.4

Дополнительные настройки производительности

Прагма	Значение	Назначение
<code>journal_mode = WAL</code>	Write-Ahead Logging	Позволяет одновременно читать и писать в БД, повышая производительность при параллельных запросах
<code>foreign_keys = ON</code>	Включено	Обеспечивает целостность ссылочных данных (при удалении пользователя удаляются его слова)
<code>busy_timeout = 5000</code>	5000 мс	При блокировке БД ожидает до 5 секунд вместо немедленной ошибки

При регистрации нового пользователя система автоматически добавляет стартовый набор из 30 слов, разбитых по 5 тематическим категориям: «Основы», «Еда», «Путешествия», «Работа», «Хобби». Это позволяет пользователю сразу начать обучение без необходимости самостоятельно добавлять слова. реализация функции добавления стартовых слов представлена в листинге 2.2.

Листинг 2.2 – Функция добавления стартовых слов (server/src/seed.js, фрагмент)

```
const insert = db.prepare(`
    INSERT INTO words (user_id, language, word, translation,
example, example_translation, category)
    VALUES (?, 'en', ?, ?, ?, ?, ?)
`);

const insertAll = db.transaction(() => {
    for (const w of words) {
        insert.run(userId, w.word, w.translation, w.example ||
'', w.example_translation || '', w.category);
    }
});

insertAll();
};
```

Функция `seedStarterWords(userId)` принимает идентификатор пользователя и вставляет 30 предопределённых слов с примерами использования и переводами примеров. Использование транзакции гарантирует, что либо все слова будут добавлены, либо ни одного.

Таблица грамматических правил создаётся и наполняется отдельным модулем `server/grammar-seed.js`. Листинг 2.3 демонстрирует фрагмент этой логики.

Листинг 2.3 – Начальное наполнение грамматических правил (server/src/grammar-seed.js, фрагмент)

```
const rules = [
    {
        language: 'en',
        title: 'Present Simple',
        description: 'Настоящее простое время. Используется для регулярных действий, фактов и привычек.',
        examples: JSON.stringify([
            { en: 'I work every day.', ru: 'Я работаю каждый день.' },
            { en: 'She lives in London.', ru: 'Она живёт в Лондоне.' },
            { en: 'They don\'t speak French.', ru: 'Они не говорят по-французски.' },
            { en: 'Do you like coffee?', ru: 'Ты любишь кофе?' }
        ]),
        category: 'tenses',
        level: 'beginner',
        order_index: 1
    },
    //... другие правила
];
```

```

const insertAll = db.transaction(() => {
  for (const r of rules) {
    insert.run(r.language,      r.title,      r.description,
r.examples, r.category, r.level, r.order_index);
  }
});
insertAll();
};

```

Перед вставкой выполняется проверка: если таблица `grammar_rules` уже содержит записи, добавление пропускается, чтобы не дублировать данные при повторных запусках сервера.

2.3 Проектирование REST API

API разработан в соответствии с принципами REST:

- Использование HTTP-методов: GET (получение), POST (создание), PUT (обновление)
- Статус-коды HTTP для индикации результата операции
- Обмен данными в формате JSON
- Аутентификация через JWT-токен в заголовке `Authorization: Bearer <token>`

Все маршруты имеют базовый префикс `/api`. Система разделена на следующие функциональные модули (таблица 2.5):

Таблица 2.5

Функциональные модули

Модуль	Базовый путь	Описание
Аутентификация	<code>/api/auth</code>	Регистрация, вход, профиль
Слова	<code>/api/words</code>	CRUD операции со словами, алгоритм повторения
Грамматика	<code>/api/grammar</code>	Справочные материалы по грамматике
Администрирование	<code>/admin</code>	Управление пользователями (только для админов)

Таблица 2.6

Эндпоинты аутентификации

Метод	Путь	Описание	Тело запроса	Ответ
POST	/api/auth/register	Регистрация нового пользователя	{username, email, password}	{token, user}
POST	/api/auth/login	Вход в систему	{email, password}	{token, user}
GET	/api/auth/profile	Получение профиля и статистики	—	{user, stats}

Таблица 2.7

Эндпоинты для работы со словами

Метод	Путь	Описание	Тело запроса	Ответ
GET	/api/words/study?language=en&limit=10	Получение слов для изучения	—	{words, stats}
PUT	/api/words/:id/review	Оценка знания слова	{quality}	{new_level, next_review_days}
POST	/api/words	Добавление нового слова	{word, translation, category, ...}	{message}
GET	/api/words/categories	Получение категорий с прогрессом	—	{categories}

Таблица 2.8

Эндпоинты для грамматики

Метод	Путь	Описание	Тело запроса	Ответ
GET	/api/grammar/categories	Получение категорий правил	—	{categories}
GET	/api/grammar?language=en&category=verbs	Получение списка правил с фильтрацией	—	{rules}
GET	/api/grammar/:id	Получение конкретного правила	—	{rule}

Регистрация пользователя

Запрос:

POST /api/auth/register
 Content-Type: application/json

```
{
  "username": "ivan",
  "email": "ivan@example.com",
  "password": "securepass123"
}
```

Ответ (успех, 201 Created):

```
{
  "message": "Регистрация успешна",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": 1,
    "username": "ivan",
    "email": "ivan@example.com"
  }
}
```

Оценка слова (алгоритм повторения)

Запрос:

PUT /api/words/15/review
 Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
 Content-Type: application/json

```
{
  "quality": 4
}
```

Ответ (успех, 200 OK):

```
{
  "success": true,
  "new_level": 3,
  "next_review_days": 7,
  "message": "Отлично!"
}
```

Получение профиля со статистикой

Запрос:

```
GET /api/auth/profile
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

Ответ (успех, 200 OK):

```
{
  "user": {
    "id": 1,
    "username": "ivan",
    "email": "ivan@example.com",
    "daily_goal": 10,
    "created_at": "2026-01-15 10:30:00",
    "stats": {
      "total_words": 45,
      "learned_words": 28,
      "words_to_review": 12,
      "completion_percent": 62,
      "by_level": [
        {"level": 0, "count": 5},
        {"level": 1, "count": 7},
        {"level": 2, "count": 5},
        {"level": 3, "count": 10},
        {"level": 4, "count": 12},
        {"level": 5, "count": 6}
      ],
      "by_category": [
        {"category": "basics", "total": 10, "learned": 7},
        {"category": "food", "total": 8, "learned": 5}
      ],
      "today_activity": {
        "reviewed": 15,
        "correct": 12
      }
    }
  }
}
```

2.4 Проектирование алгоритма интервального повторения

На рис. 2.3 представлена блок – схема алгоритма интервального повторения.

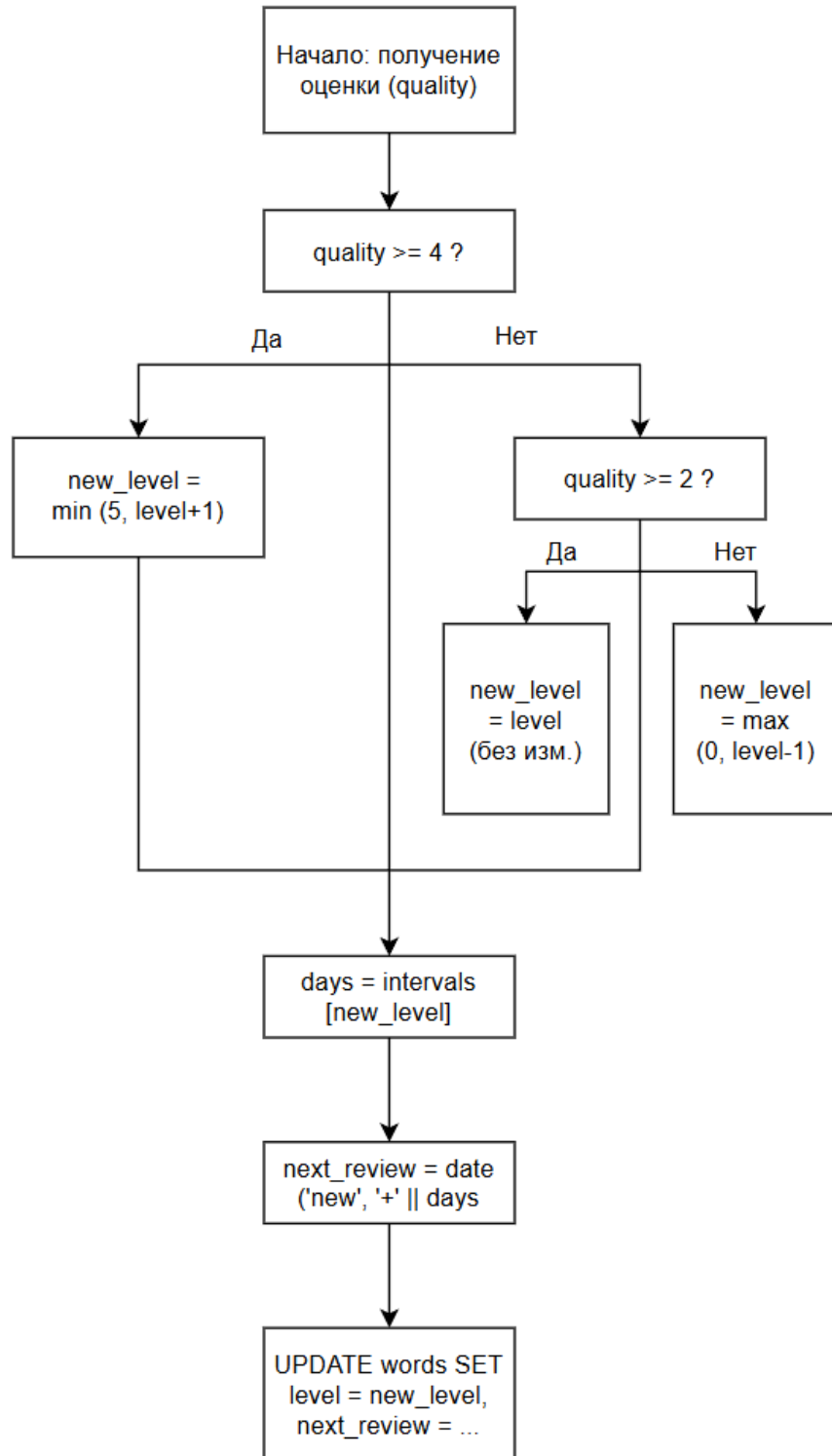


Рис. 2.3 – Блок-схема алгоритма интервального повторения

Блок 1 (Начало). Алгоритм запускается после того, как пользователь нажал одну из четырёх кнопок оценки ($quality = 0, 2, 4$ или 5) на карточке слова. На вход подаётся текущий уровень слова ($level$) и полученная оценка ($quality$).

Блок 2 (Проверка $quality \geq 4$). Первое условное ветвление проверяет, является ли оценка высокой («Хорошо» или «Отлично»). Если условие истинно, уровень слова повышается на 1 ($new_level = level + 1$), но не выше максимального значения 5. Это соответствует успешному запоминанию слова.

Блок 3 (Проверка $quality \geq 2$). Если оценка не является высокой, алгоритм проверяет, является ли она нейтральной («Не очень», $quality = 2$). При истинности этого условия уровень слова остаётся неизменным ($new_level = level$). Это означает, что слово не требует немедленного повторения, но и не считается полностью выученным.

Блок 4 ($quality < 2$). Если оба предыдущих условия ложны, значит, оценка является низкой («Сложно», $quality = 0$). В этом случае уровень слова понижается на 1 ($new_level = level - 1$), но не ниже минимального значения 0. Это заставляет систему показать слово раньше, так как интервал сократится.

Блок 5 (Определение интервала). После вычисления нового уровня алгоритм определяет соответствующий ему интервал в днях, используя фиксированный массив: $intervals = [0, 1, 3, 7, 14, 30]$. Например, для $new_level = 3$ интервал составит 7 дней.

Блок 6 (Расчёт даты следующего повторения). Новая дата следующего повторения вычисляется по формуле: $next_review = текущая_дата + daysUntilReview$. В коде это реализовано с помощью SQLite-функции `date('now', '+' || days || ' days')`.

Блок 7 (Обновление базы данных). Завершающий этап — выполнение SQL-запроса `UPDATE`, который сохраняет новый уровень, новую дату повторения и увеличивает счётчик повторений ($review_count$).

Назначение алгоритма.

Такая последовательность действий обеспечивает автоматическое адаптивное планирование повторений: чем лучше пользователь помнит слово, тем реже оно показывается, и наоборот — при ошибке слово возвращается на более ранний уровень, что соответствует принципам интервального повторения.

Интервалы между повторениями определяются уровнем слова и представлены в таблице 2.9.

Таблица 2.9

Зависимость интервала от уровня

Уровень (level)	Интервал (дней)	Значение для обучения
0	0	Немедленное повторение (при ошибке)
1	1	Ежедневное повторение для новых слов
2	3	Переход на более редкие повторения
3	7	Еженедельное повторение
4	14	Двухнедельное повторение
5	30	Ежемесячное повторение (выучено надолго)

В листинге 2.4 представлена логика изменения уровня слова в зависимости от полученной оценки.

Листинг 2.4 – Реализация алгоритма интервального повторения (server/src/routes/words.js)

```
// Алгоритм интервального повторения
let newLevel = word.level;
if (quality >= 4) {
```

```
        newLevel = Math.min(5, word.level + 1); // Повышаем
    } else if (quality >= 2) {
        newLevel = word.level; // Оставляем
    } else {
        newLevel = Math.max(0, word.level - 1); // Понижаем
    }
}
```

При оценке 4 или 5 уровень повышается на 1 (но не выше 5). При оценке 2 уровень не меняется. При оценке 0 уровень понижается на 1 (но не ниже 0). После определения нового уровня из массива [0, 1, 3, 7, 14, 30] выбирает интервал до следующего повторения.

2.5 Выбор и обоснование технологий

Backend: Node.js + Express.js

Выбор Node.js в качестве платформы для серверной части обусловлен следующими факторами:

1. Единый язык с фронтендом – и клиентская, и серверная части написаны на JavaScript, что упрощает разработку и поддержку
2. Асинхронная модель – неблокирующая архитектура позволяет эффективно обрабатывать множество одновременных запросов
3. Богатая экосистема – наличие большого количества готовых модулей (npm) ускоряет разработку

Express.js выбран как наиболее популярный и легковесный фреймворк для Node.js, предоставляющий удобный маршрутизатор и middleware-механизм.

База данных: SQLite

SQLite выбран по следующим причинам:

1. Отсутствие отдельного сервера – база данных хранится в одном файле app.db, что упрощает развёртывание и перенос приложения
2. ACID-совместимость – поддержка транзакций и целостности данных

3. Достаточная производительность – для дипломного проекта с небольшим количеством пользователей SQLite является оптимальным решением
4. Библиотека better-sqlite3 – обеспечивает синхронный API, что упрощает написание кода по сравнению с асинхронными аналогами

Frontend: React + Vite

React выбран как одна из самых популярных библиотек для создания пользовательских интерфейсов. Ключевые преимущества:

- Декларативный подход – описание того, как интерфейс должен выглядеть в зависимости от состояния
- Компонентная архитектура – переиспользование кода
- Виртуальный DOM – эффективное обновление интерфейса

Vite выбран в качестве сборщика вместо устаревшего Webpack, так как обеспечивает:

- Мгновенный холодный старт
- Быструю горячую перегрузку модулей (HMR)
- Оптимизированную сборку для продакшена

Аутентификация: JWT

JSON Web Token (JWT) выбран как стандартный механизм аутентификации для SPA-приложений:

1. Отсутствие состояния – сервер не хранит информацию о сессиях, что упрощает масштабирование
2. Безопасность – токен подписывается секретным ключом, что предотвращает подделку
3. Удобство хранения – токен хранится в localStorage клиента и автоматически подставляется в заголовки запросов

Автоматический перевод: MyMemore API

Для автоматического перевода слов при добавлении используется бесплатный REST API memory.translated.net. Преимущества выбора:

- Поддержка множества языковых пар
- Достаточное качество перевода для учебных целей

3 Реализация программного обеспечения

3.1 Организация серверной части (Backend)

Серверная часть построена на платформе Node.js с использованием фреймворка Express.js. Проект имеет следующую структуру каталогов (рис. 3.1).

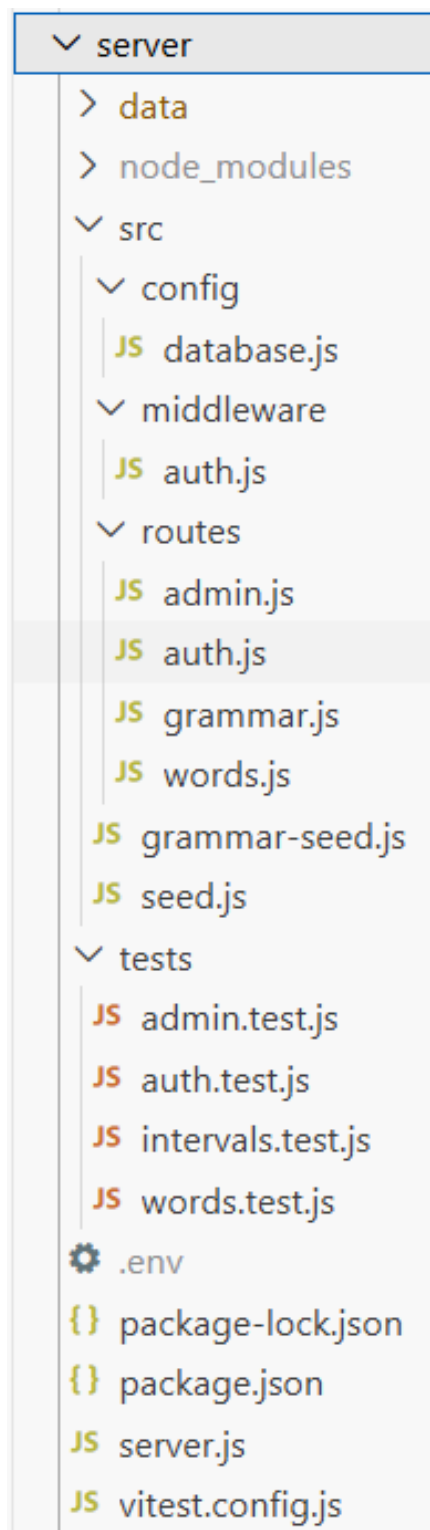


Рис.3.1 – Структура каталогов серверной части

Файл `server.js` является главным файлом сервера. Он инициализирует Express-приложение, подключает `middleware`, регистрирует маршруты и запускает сервер на прослушивание. В листинге 3.1 представлена точка входа сервера.

Листинг 3.1 – Точка входа сервера (`server/server.js`)

```

const app = express();
const PORT = process.env.PORT || 5000;
const __dirname = path.dirname(fileURLToPath(import.meta.url));

app.use(cors({
  origin: ['http://localhost:5173', 'http://127.0.0.1:5173'],
  credentials: true
}));
app.use(express.json());

// API маршруты
app.use('/api/auth', authRoutes);
app.use('/api/words', wordRoutes);
app.use('/api/grammar', grammarRoutes);

app.listen(PORT, () => {console.log(`

Сервер: http://localhost:${PORT}
Админка: http://localhost:${PORT}/admin
API: http://localhost:${PORT}/api

`);
  // Заполняем грамматику после запуска
  seedGrammarRules();
});

```

Сервер инициализируется на порту 5000. Подключаются middleware CORS (для работы с фронтендом на порту 5173) и JSON-парсер. Регистрируются маршруты аутентификации, работы со словами и грамматики.

Middleware auth.js отвечает за проверку JWT-токена во входящих запросах. Все защищённые маршруты (работа со словами, профиль, статистика) используют этот middleware.

Листинг 3.2 – Проверка JWT - токена (server/src/middleware/auth.js, фрагмент)

```

const token = authHeader.split(' ')[1];
const decoded = jwt.verify(token, JWT_SECRET);
const user = db.prepare('SELECT id, role FROM users WHERE id = ?').get(decoded.userId);
if (!user) {
  return res.status(401).json({ error: 'Пользователь не найден' });
}
next();

```

Middleware извлекает токен из заголовка Authorization, удаляя префикс Bearer. Затем с помощью библиотеки jsonwebtoken проверяется

подлинность токена и его срок действия. Если токен действителен, из него извлекается идентификатор пользователя, после чего выполняется запрос к базе данных для подтверждения существования пользователя. При успешной проверке идентификатор пользователя сохраняется в объекте запроса (req.userId) для использования в последующих обработчиках маршрутов. В случае ошибки (отсутствие токена, неверная подпись, истекший срок или отсутствие пользователя) возвращается ответ с кодом 401.

Маршруты auth.js реализуют регистрацию и вход пользователей, а также получение профиля с расширенной статистикой. В листинге 3.3 представлен фрагмент обработчика регистрации.

Листинг 3.3 – Регистрация пользователя (server/src/routes/auth.js, фрагмент)

```
const { username, email, password } = req.body;

if (password.length < 6) {
  return res.status(400).json({ error: 'Пароль минимум 6 символов' });
}

const passwordHash = bcrypt.hashSync(password, 10);
const result = db.prepare(
  'INSERT INTO users (username, email, password_hash) VALUES (?, ?, ?)'
).run(username, email, passwordHash);

// Создаем токен
const token = jwt.sign({ userId }, JWT_SECRET, { expiresIn: '30d' });

res.status(201).json({
  message: 'Регистрация успешна',
  token,
  user: { id: userId, username, email }
});
```

При регистрации сервер сначала проверяет длину пароля (не менее 6 символов), а также наличие имени и email. Затем пароль хешируется с помощью библиотеки bcrypt (10 раундов соления) для безопасного хранения в базе данных. После создания записи в таблице users генерируется JWT-токен со сроком действия 30 дней, который возвращается клиенту вместе с данными нового пользователя. При успешной регистрации также автоматически вызывается функция seedStarterWords(), добавляющая стартовый набор из 30 слов для изучения. Аналогично реализован маршрут

входа, который проверяет email и сравнивает пароль с хешем, а маршрут получения профиля возвращает расширенную статистику (количество слов, прогресс по уровням, активность за сегодня).

3.2 Организация клиентской части (Frontend)

Клиентская часть построена на React с использованием Vite в качестве сборщика.

Проект имеет структуру, представленную на рис. 3.2.

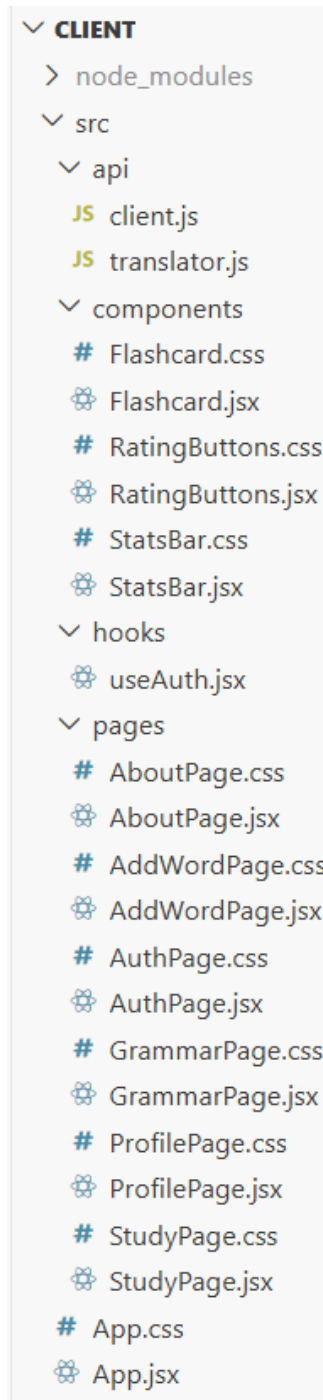


Рис. 3.2 – Структура каталогов клиентской части

Клиент `client.js` инкапсулирует все HTTP – запросы к серверу, автоматически подставляет JWT – токен и обрабатывает ответы. В листинге 3.4 представлена базовая структура этого модуля.

Листинг 3.4 – API – клиент (`client/src/api/client.js`)

```
async request(endpoint, options = {}) {
  const config = {
    headers: {
      'Content-Type': 'application/json',
      ...(this.token && { Authorization: `Bearer
${this.token}` })),
    },
    ...options,
  };

  const response = await fetch(`${this.baseURL}${endpoint}`,
config);
  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.error || 'Ошибка запроса');
  }

  return data;
}
```

Метод `request()` является ядром API-клиента. Он формирует HTTP-запрос, автоматически добавляя в заголовок `Authorization` JWT-токен, если тот сохранён в локальном хранилище. Все методы клиента (`register`, `login`, `getProfile`, `getWordsForStudy`, `reviewWord`, `addWord` и другие) используют этот универсальный метод, передавая ему `endpoint`, HTTP-метод и тело запроса. При получении ответа клиент проверяет статус: если ответ успешный, возвращаются данные в формате JSON; если нет — выбрасывается исключение с сообщением об ошибке, которое можно обработать в компонентах React. Такой подход централизует логику работы с сетью, упрощает отладку и обеспечивает единообразную обработку ошибок во всём приложении. Токен сохраняется в `localStorage` при входе или регистрации и удаляется при выходе из системы.

Компонент `StudyPage` является основной страницей приложения. Он управляет состоянием карточек, отображает текущее слово, обрабатывает

оценку и обновляет статистику. В листинге 3.5 представлены ключевые функции этого компонента.

Листинг 3.5 – Главная страница изучения
(client/src/pages/StudyPage.jsx, фрагмент)

```
const loadWords = async (limit) => {
  try {
    const count = limit || wordsPerDay;
    console.log('Загружаем слова, лимит:', count);
    const data = await api.getWordsForStudy('en', count);
    console.log('Получено слов:', data.words.length);
    setWords(data.words);
    if (data.words.length > 0) {
      setCurrentWord(data.words[0]);
      setShowAnswer(false);
    } else {
      setCurrentWord(null);
    }
  } catch (err) {
    console.error('Load words error:', err);
  }
};

const handleReview = async (quality) => {
  await api.reviewWord(currentWord.id, quality);
  const remaining = words.slice(1);
  // Получаем актуальную статистику из API
  const wordsData = await api.getWordsForStudy('en', 100);
  const profileData = await api.getProfile();
  setStats({
    total: profileData.user.stats.total_words || 0,
    learned: profileData.user.stats.learned_words || 0,
    due: wordsData.words.length,
  });
  if (remaining.length > 0) {
    setWords(remaining);
    setCurrentWord(remaining[0]);
    setShowAnswer(false);
  } else {
    setWords([]);
    setCurrentWord(null);
  }
};
```

Функция `loadWords()` загружает с сервера слова для изучения с учётом выбранного пользователем лимита (по умолчанию 10 слов). После получения данных первое слово устанавливается как текущее, а флаг `showAnswer` сбрасывается, чтобы карточка отображала вопрос, а не ответ.

Функция `handleReview()` вызывается при нажатии пользователем одной из кнопок оценки. Она отправляет на сервер оценку `quality` (0, 2, 4 или 5), после чего из массива слов удаляется текущее слово, а оставшиеся сдвигаются. Затем выполняются два параллельных запроса для обновления статистики: получение актуального списка слов к повторению и получение профиля с расширенной статистикой. После обновления состояния компонента отображается следующее слово. Если все слова на сегодня повторены, `currentWord` устанавливается в `null`, и пользователь видит сообщение о завершении с анимированной иконкой.

Компонент `Flashcard` отображает карточку слова. При нажатии на карточку показывается перевод и пример использования. В листинге 3.6 представлена структура этого компонента.

Листинг 3.6 – Отображение карточки слова
(`client/src/components/Flashcard.jsx`, фрагмент)

```
<div className="word-display">
  {showAnswer ? word.translation : word.word}
</div>
{showAnswer && word.example && (
  <div className="example-block">
    <div className="example-original"> □ {word.example}</div>
    {word.example_translation && (
      <div className="example-translation">RU: {word.example_translation}</div>
    )}
  )}
)}
```

В зависимости от значения пропса `showAnswer` компонент отображает либо слово на иностранном языке (режим вопроса), либо его перевод (режим ответа). Если включён режим ответа и для слова указан пример использования, под переводом показывается пример на иностранном языке и его перевод на русский. Также на карточке отображается категория слова с соответствующим эмодзи («Основы», «Еда», «Путешествия», «Работа», «Хобби») и индикатор уровня изученности в виде звёзд (количество звёзд соответствует текущему уровню слова от 0 до 5). При нажатии на карточку вызывается переданный через пропсы обработчик `onClick`, который

переключает режим отображения. Компонент оформлен с затемнёнными углами и анимацией при наведении, что улучшает взаимодействие пользователя с интерфейсом.

Компонент RatingButtons предоставляет пользователю четыре варианта оценки запоминания. Каждой кнопке соответствует числовое значение quality, которое отправляется на сервер. В листинге 3.7 представлена структура этого компонента.

Листинг 3.7 – Кнопки оценки (client/src/components/RatingButtons.jsx, фрагмент)

```
const ratings = [
  {quality: 0, label: 'Сложно', emoji: '🔴', className: 'again'},
  {quality: 2, label: 'Не очень', emoji: '🟡', className: 'hard'},
  {quality: 4, label: 'Хорошо', emoji: '🟢', className: 'good' },
  {quality: 5, label: 'Отлично', emoji: '🟠', className: 'easy' },
];
{ratings.map(({ quality, label, emoji, className }) => (
  <button
    key={quality}
    onClick={() => onRate(quality)}
    className={`rate-btn ${className}`}
  >
    <span className="rate-emoji">{emoji}</span>
    <span className="rate-label">{label}</span>
  </button>
))}
```

Компонент принимает через пропсы функцию onRate, которая вызывается при нажатии на любую из кнопок. Массив ratings содержит четыре объекта, каждый из которых определяет внешний вид и поведение одной кнопки: числовое значение quality (0, 2, 4 или 5), текстовую подпись, эмодзи и CSS-класс для цветового оформления. При нажатии на кнопку вызывается onRate с соответствующим значением quality, которое затем отправляется на сервер для обновления уровня слова.

Каждая кнопка имеет уникальный цвет: красный для «Сложно», жёлтый для «Не очень», сине-фиолетовый для «Хорошо» и зелёный для «Отлично». При наведении курсора кнопка слегка поднимается вверх, обеспечивая визуальную обратную связь. На мобильных устройствах кнопки автоматически перестраиваются в сетку 2×2 для удобства нажатия пальцем, а

на десктопе располагаются в один ряд. Такая цветовая кодировка помогает пользователю быстро идентифицировать нужную оценку без чтения текста.

Компонент StatsBar отображает прогресс обучения: общее количество слов, выученные слова и слова, ожидающие повторения. В листинге 3.8 представлена структура этого компонента.

Листинг 3.8 – Панель статистики (client/src/components/StatsBar.jsx, фрагмент)

```
const items = [
  { label: 'Всего слов', value: stats.total || 0, icon: '□' },
  { label: 'Изучено', value: stats.learned || 0, icon: '□' },
  { label: 'На сегодня', value: stats.due || 0, icon: '□' },
];

{items.map(({ label, value, icon }) => (
  <div key={label} className="stat-item">
    <span className="stat-icon">{icon}</span>
    <div className="stat-info">
      <span className="stat-value">{value}</span>
      <span className="stat-label">{label}</span>
    </div>
  </div>
))}
```

Компонент принимает через пропсы объект stats, содержащий три числовых поля: total (общее количество слов в словаре пользователя), learned (количество выученных слов — уровень 3 и выше) и due (количество слов, запланированных для повторения на сегодня). Массив items определяет три отображаемых показателя, каждый из которых включает текстовую подпись, значение и соответствующую эмодзи (для общего числа, для изученных, для слов к повторению).

При отсутствии данных (например, во время загрузки) используются значения по умолчанию — 0, что предотвращает ошибки рендеринга. Компонент не содержит собственной логики вычислений — только отображение переданных данных. Такой подход обеспечивает его переиспользуемость и простоту тестирования. Статистика обновляется после каждой оценки слова, после добавления нового слова и при

переключении между страницами. Визуально панель представляет собой горизонтальный ряд из трёх карточек на десктопе, которые на мобильных устройствах перестраиваются вертикально или в несколько рядов для сохранения читаемости.

3.3 Интеграция с внешним API перевода

Для автоматического перевода слов при добавлении используется бесплатный REST API MyMemory Translate (<https://api.mymemory.translated.net>). Это позволяет пользователю ввести слово на иностранном языке и получить автоматический перевод без ручного ввода. В листинге 3.9 представлена функция, реализующая вызов этого API.

Листинг 3.9 – Функция перевода слов (client/src/api/translator.js)

```
export async function translateWord(word, from = 'auto', to = 'ru') {
  try {
    let sourceLang = from;
    if (sourceLang === 'auto') {
      const isRussian = /[а-яё]/i.test(word);
      sourceLang = isRussian ? 'ru' : 'en';
    }
    const res = await fetch(
      `https://api.mymemory.translated.net/get?q=${encodeURIComponent(
        word
      )}&langpair=${sourceLang}|${to}`
    );
    const data = await res.json();
    if (data.responseStatus === 200) {
      return data.responseData.translatedText;
    }
    return null;
  } catch {
    return null;
  }
}
```

Функция `translateWord()` принимает три параметра: слово для перевода (`word`), язык источника (`from`) и язык назначения (`to`). Если язык источника не указан или передан как `'auto'`, функция автоматически определяет его по наличию кириллических символов: если в слове

встречаются русские буквы, язык источника устанавливается как 'ru', иначе — 'en'. Это позволяет пользователю вводить слово на любом языке без необходимости переключать режим.

Затем формируется GET-запрос к API MyMemory Translate. Параметр `q` содержит исходное слово, закодированное для безопасной передачи в URL, а `langpair` задаёт пару языков в формате источник|назначение. После получения ответа проверяется статус: если `responseStatus === 200` (успешный запрос), функция возвращает переведённый текст; в противном случае возвращается `null`. В компоненте `AddWordPage` эта функция вызывается при нажатии на кнопку перевода рядом с полями «Слово» и «Пример использования». Во время выполнения запроса отображается анимация загрузки, а полученный перевод автоматически подставляется в соответствующее поле формы.

3.4 Стилизация и адаптивный дизайн

В разработанном приложении используется модульный подход к стилизации: для каждого компонента React создаётся отдельный CSS-файл. Это обеспечивает изолированность стилей, предотвращает конфликты имён классов и упрощает поддержку кода при масштабировании проекта.

Главная страница изучения оформлена в тёмной теме, что снижает нагрузку на зрение при длительном использовании приложения. Основные стилистические решения представлены в таблице 3.1.

Таблица 3.1

Основные стилистические решения StudyPage.css

Элемент	Свойства	Назначение
Фон страницы	<code>min-height: 100vh; padding: 20px</code>	Полноэкранный режим с отступами

Шапка страницы	max-width: 500px; margin: 0 auto; background: #1a1a2e	Центрирование, тёмный фон
Карточка завершения	animation: bounce 1s ease infinite	Пульсация иконки
Кнопки настройки	.btn-count.active	Активное состояние выбранного лимита

Продолжение табл. 3.1

Адаптивность	@media (max-width: 480px)	Мобильная версия
--------------	------------------------------	------------------

Для стилизации главной страницы используется отдельный CSS – файл, реализующий тёмную тему и адаптивный дизайн. В листинге 3.10 представлены основные стили этого компонента.

Листинг 3.10 – Стили главной страницы
(client/src/pages/StudyPage.css, фрагмент)

```
.study-page {
  min-height: 100vh;
  padding: 20px;
}

.study-header {
  max-width: 500px;
  margin: 0 auto 20px;
  background: #1a1a2e;
  border: 1px solid #2a2a4a;
  border-radius: 20px;
  padding: 20px;
}

.btn-profile {
  padding: 8px 16px;
  background: #7c8aff;
  color: white;
  border: none;
  border-radius: 8px;
  cursor: pointer;
  font-size: 14px;
  white-space: nowrap;
}
```

```

.complete-icon {
  font-size: 4rem;
  margin-bottom: 16px;
  animation: bounce 1s ease infinite;
}

@keyframes bounce {
  0%,
  100% {
    transform: translateY(0);
  }
  50% {
    transform: translateY(-15px);
  }
}

.btn-count.active {
  background: #7c8aff;
  color: white;
  border-color: #7c8aff;
}

```

Фон страницы занимает всю высоту экрана (`min-height: 100vh`) с отступами по краям, что обеспечивает корректное отображение на любых устройствах независимо от размера контента. Шапка страницы центрируется, имеет максимальную ширину 500 пикселей, тёмный фон (`#1a1a2e`) и скруглённые углы 20 пикселей, что создаёт эффект плавающей карточки над основным фоном.

Кнопки профиля и навигации выполнены в акцентном цвете `#7c8aff` с белым текстом и скруглением 8 пикселей. При наведении они меняют оттенок на более тёмный (`#6670f0`), обеспечивая визуальную обратную связь. Активная кнопка выбора количества слов (5, 10, 15, 20) подсвечивается тем же акцентным цветом, что помогает пользователю видеть текущую настройку — остальные кнопки выглядят приглушёнными (серый фон и текст).

Анимация `bounce` создаёт эффект пульсации для иконки завершения изучения — она плавно поднимается вверх на 15 пикселей и возвращается обратно. Анимация зациклена и длится 1 секунду, привлекая внимание пользователя к сообщению об успешном завершении сессии. На

мобильных устройствах с шириной экрана менее 480 пикселей отступы и размер шрифта уменьшаются, шапка перестраивается в вертикальное расположение, а кнопки становятся компактнее для комфортного использования на маленьком экране.

Компонент `RatingButtons` предоставляет пользователю четыре кнопки для оценки запоминания слова. В листинге 3.11 предоставлены стили, определяющие внешний вид этих кнопок.

Листинг 3.11 — Стили кнопок оценки
(client/src/components/RatingButtons.css, фрагмент)

```
.rating-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
}

.rate-btn {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 4px;
  padding: 16px;
  border: none;
  border-radius: 16px;
  cursor: pointer;
  transition: transform 0.2s, box-shadow 0.2s;
  font-weight: bold;
}

.rate-btn:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.3);
}

.rate-btn.again {background: #ff4757; color: white;}
.rate-btn.hard {background: #ffa502; color: #1a1a2e;}
.rate-btn.good {background: #7c8aff; color: white;}
.rate-btn.easy {background: #2ed573; color: #1a1a2e;}
```

Кнопки размещаются с помощью CSS Grid, образуя сетку из двух колонок. Такое расположение обеспечивает удобство использования как на десктопе, так и на мобильных устройствах. Каждая кнопка оформлена как гибкий контейнер (`display: flex`) с вертикальным расположением элементов — эмодзи сверху, текст снизу. Кнопки имеют скруглённые углы (16 пикселей) и отступы для комфортного нажатия пальцем.

При наведении курсора кнопка слегка поднимается вверх (сдвиг на 2 пикселя) и приобретает тень, создавая эффект парения и обеспечивая визуальную обратную связь. Цветовая кодировка помогает пользователю быстро идентифицировать нужную оценку без чтения текста: красный (#ff4757) для «Сложно», жёлтый (#ffa502) для «Не очень», сине-фиолетовый (#7c8aff) для «Хорошо» и зелёный (#2ed573) для «Отлично». Для кнопок с тёмным фоном (жёлтая и зелёная) используется тёмный цвет текста для лучшей читаемости, для красной и синей — белый.

Для обеспечения комфортного использования приложения на мобильных устройствах с небольшой диагональю экрана реализованы медиа-запросы. В листинге 3.12 представлены адаптивные стили, применяемый при ширине экрана 480 пикселей.

Листинг 3.12 – Адаптивные стили (client/src/pages/StudyPage/css, фрагмент)

```
@media (max-width: 480px) {  
  .study-page {padding: 10px;}  
  .study-header {padding: 15px; border-radius: 16px;}  
  .header-top {flex-direction: column; align-items: flex-start; gap:  
10px;}  
  .header-top h1 {font-size: 1.1rem;}  
  .header-buttons {width: 100%; justify-content: flex-start; gap:  
5px;}  
  .btn-profile, .btn-logout {padding: 6px 8px; font-size: 11px;}  
  .flashcard {padding: 30px 20px; border-radius: 16px;}  
  .word-display {font-size: 1.8rem;}  
  .rating-grid {grid-template-columns: 1fr 1fr; gap: 8px;}  
  .rate-btn {padding: 12px 10px; font-size: 13px;}  
  .complete-card {padding: 40px 20px;}  
}
```

При ширине экрана менее 480 пикселей (что соответствует большинству современных смартфонов в портретной ориентации) внешние отступы страницы уменьшаются с 20 до 10 пикселей, позволяя эффективнее использовать ограниченное пространство экрана. Шапка страницы получает меньшие внутренние отступы (15 пикселей вместо 20) и уменьшенное скругление углов (16 пикселей вместо 20).

Верхняя часть шапки перестраивается с горизонтального расположения элементов на вертикальное: заголовок приветствия, кнопки навигации и настройки выстраиваются в колонку. Размер шрифта заголовка уменьшается до 1.1 rem, а кнопки становятся компактнее — отступы уменьшаются до 6×8 пикселей, размер шрифта до 11 пикселей. Это предотвращает перенос текста и сохраняет читаемость на маленьком экране.

Карточка слова также адаптируется: внутренние отступы уменьшаются с 60×40 до 30×20 пикселей, скругление углов — с 30 до 16 пикселей. Размер отображаемого слова снижается до 1.8 rem, что удобно для чтения без горизонтальной прокрутки. Сетка кнопок оценки сохраняет структуру 2×2, но уменьшаются отступы между кнопками (с 10 до 8 пикселей) и их внутренние поля (с 16 до 12×10 пикселей), а шрифт становится мельче (13 пикселей вместо 14). Сообщение о завершении получает более компактные отступы (40×20 вместо 60×30 пикселей), чтобы вся информация помещалась на экране без необходимости прокрутки. Все эти изменения обеспечивают комфортное использование приложения на мобильных устройствах без потери функциональности и читаемости.

Тёмная тема (#1a1a2e с текстом #e0e0e0) выбрана по следующим причинам:

1. Снижение утомляемости глаз – при длительном изучении слов яркий белый фон быстрее утомляет, особенно в вечернее время.
2. Акцент на контенте – цветные кнопки оценок лучше выделяются на тёмном фоне.
3. Современный тренд – многие современные приложения (Duolingo, Anki, Quizlet) предлагают тёмную тему как опцию или используют её по умолчанию.

Акцентный цвет #7c8aff (приглушённый сине-фиолетовый) выбран как:

- Достаточно яркий для привлечения внимания

- Не раздражающий при длительном использовании
- Хорошо сочетающийся с тёмным фоном

4 Тестирование и анализ результатов

4.1 Цели и задачи тестирования

После завершения разработки веб – приложения «Интерактивный тренажёр лексики» было проведено функциональное тестирование с целью проверки соответствия разработанной системы требованиям, сформулированным в главе 1.

Основные задачи тестирования:

1. Проверить работоспособность всех функциональных требований (FR01 – FR07)
2. Убедиться в корректной работе алгоритма интервального повторения
3. Проверить обработку ошибок и краевых случаев
4. Оценить удобство пользовательского интерфейса
5. Проверить адаптивность вёрстки на различных устройствах
6. Выполнить автоматическое модульное тестирование ключевых компонентов

4.2 Сценарии функционального тестирования

Для проверки работоспособности системы были разработаны тестовые сценарии, охватывающие все ключевые функции приложения. Каждый сценарий включает описание входных данных, ожидаемый результат и фактический результат.

В таблице 4.1 представлены сценарии проверки регистрации и входа пользователей, включая валидацию полей, обработку ошибок и защиту маршрутов.

Таблица 4.1

Тестирование регистрации и входа

ID	Сценарий	Действие	Ожидаемый результат	Фактический результат
TO1	Успешная регистрация	Ввод username = “testes”, email = “tester@test.com, password = “123456”	Статус 201, получен JWT – токен, создан пользователь	Статус 201, получен JWT – токен, создан пользователь
TO2	Регистрация с коротким именем	username = “ab” (менее 3 символов)	Ошибка: «Имя пользователя минимум 3 символа»	Ошибка: «Имя пользователя минимум 3 символа»
TO3	Регистрация с коротким паролем	password = “12345” (менее 6 символов)	Ошибка: «Пароль минимум 6 символов»	Ошибка: «Пароль минимум 6 символов»
TO4	Регистрация с некорректным email	email = “test@”	Ошибка: «Некорректный email»	Ошибка: «Некорректный email»
TO5	Регистрация с существующим email	Повторная регистрация с email = “tester@test.com”	Ошибка: «Email или имя уже заняты»	Ошибка: «Email или имя уже заняты»
TO6	Успешный вход	email = “tester@test.com”, password = “123456”	Статус 200, получен JWT – токен, данные пользователя	Статус 200, получен JWT – токен, данные пользователя
TO7	Вход с неверным паролем	email = “tester@test.com”, password = “wrong”	Ошибка: «Неверный email или пароль»	Ошибка: «Неверный email или пароль»
TO8	Доступ к защищённому API	Запрос GET /api/words/study без	Ошибка 401: «Требуется	Ошибка 401: «Требуется

	без токена	заголовка Authorization	авторизация»	авторизация»
Т09	Доступ с недействительным токеном	Запрос с поддельным JWT - токеном	Ошибка 401: «Токен недействителен или истёк»	Ошибка 401: «Токен недействителен или истёк»

Все сценарии аутентификации успешно пройдены. Система корректно обрабатывает как успешные, так и ошибочные сценарии, обеспечивая безопасность доступа к защищённым маршрутам.

В таблице 4.2 представлены сценарии проверки операций со словами: получение слов для изучения, оценка знания, добавление новых слов и получение категорий.

Таблица 4.2

Тестирование операций со словами

ID	Сценарий	Действие	Ожидаемый результат	Фактический результат
T10	Получение слов для изучения	GET /api/words/study?limit=10	Массив до 10 слов, требующих повторения	Массив до 10 слов, требующих повторения
T11	Получение статистики одновременно со словами	Проверка поля stats в ответе	Объект с total, learned, due	Объект с total, learned, due
T12	Оценка слова «Отлично» (quality=5)	Уровень слова был 2 → оценка 5	Уровень становится 3, интервал 7 дней	Уровень становится 3, интервал 7 дней
T13	Оценка слова «Хорошо» (quality=4)	Уровень слова был 2 → оценка 4	Уровень становится 3, интервал 7 дней	Уровень становится 3, интервал 7 дней
T14	Оценка слова «Не очень» (quality=2)	Уровень слова был 2 → оценка 2	Уровень остаётся 2, интервал 3 дня	Уровень остаётся 2, интервал 3 дня
T15	Оценка слова «Сложно» (quality=0)	Уровень слова был 2 → оценка 0	Уровень становится 1, интервал 1 день	Уровень становится 1, интервал 1 день
T16	Оценка слова на максимальном уровне	Уровень слова был 5 → оценка 5	Уровень остаётся 5, интервал 30	Уровень остаётся 5, интервал 30

			дней	дней
T17	Оценка слова на минимальном уровне	Уровень слова был 0 → оценка 0	Уровень остаётся 0, интервал 0 дней	Уровень остаётся 0, интервал 0 дней
T18	Добавление нового слова (ручной ввод)	word="cat", translation="кот"	Слово сохранено, уровень=0, интервал=0	Слово сохранено, уровень=0, интервал=0

Продолжение табл. 4.2

T19	Добавление слова с автопереводом	Ввод word="dog", нажатие кнопки перевода	Поле перевода автоматически заполняется «собака»	Поле перевода автоматически заполняется «собака»
T20	Добавление слова с пустыми полями	word="", translation=""	Ошибка: «Слово и перевод обязательны»	Ошибка: «Слово и перевод обязательны»
T21	Добавление дубликата слова	Повторное добавление слова «cat»	Ошибка: «Такое слово уже существует»	Ошибка: «Такое слово уже существует»
T22	Получение категорий с прогрессом	GET /api/words/categories	Список категорий с полями total и learned	Список категорий с полями total и learned

Все сценарии работы со словами успешно пройдены. Алгоритм интервального повторения корректно изменяет уровень в зависимости от оценки. Автоматический перевод успешно заполняет поле перевода. Защита от дубликатов предотвращает создание одинаковых слов.

В таблице 4.3 представлены сценарии проверки получения профиля пользователя и расширенной статистики.

Таблица 4.3

Тестирование профиля и статистики

ID	Сценарий	Действие	Ожидаемый	Фактический
----	----------	----------	-----------	-------------

			результат	результат
T23	Получение профиля	GET /api/auth/profile	Данные пользователя + объект stats	Данные пользователя + объект stats
T24	Проверка количества слов	B stats.total_words	Соответствует количеству слов в БД	Соответствует количеству слов в БД
T25	Проверка выученных слов	B stats.learned_words	Количество слов с уровнем ≥ 3	Количество слов с уровнем ≥ 3

Продолжение табл. 4.3

T26	Проверка слов к повторению	B stats.words_to_review	Количество слов с $\text{next_review} \leq \text{now}()$	Количество слов с $\text{next_review} \leq \text{now}()$
T27	Проверка разбивки по уровням	B stats.by_level	Массив объектов {level, count}	Массив объектов {level, count}
T28	Проверка разбивки по категориям	B stats.by_category	Массив объектов {category, total, learned}	Массив объектов {category, total, learned}
T29	Проверка дневной активности	B stats.today_activity	review и correct за сегодня	review и correct за сегодня

Все сценарии статистики успешно пройдены. Профиль возвращает корректные данные пользователя, а статистика правильно отражает прогресс изучения.

В таблице 4.4 представлены сценарии проверки отображения интерфейса на различных устройствах и корректности навигации.

Таблица 4.4

Тестирование UI и адаптивности

ID	Сценарий	Устройство / разрешение	Ожидаемый результат	Фактический результат
T30	Отображение на десктопе	1920×1080	Все элементы видны, сетка 1×4 кнопок	Все элементы видны, сетка 1×4 кнопок
T31	Отображение на	768×1024	Все элементы	Все элементы

	планшете		видны, сетка 1×4 кнопок	видны, сетка 1×4 кнопок
T32	Отображение на мобильном	375×667	Горизонтальная прокрутка отсутствует, сетка 2×2 кнопок	Горизонтальная прокрутка отсутствует, сетка 2×2 кнопок
T33	Переключение количества слов	Выбор 5/10/15/20 слов	Обновляется лимит, загружается соответствующее количество	Обновляется лимит, загружается соответствующее количество
T34	Карточка слов	Клик по карточке	Показывает перевод и пример	Показывает перевод и пример

Продолжение табл. 4.4

T35	Анимация завершения	После изучения всех слов	Появляется иконка с анимацией bounce	Появляется иконка с анимацией bounce
T36	Навигация	Переход на страницы Профиль/Грамматика/Добавить слово	Корректное переключение между страницами	Корректное переключение между страницами

Все сценарии UI успешно пройдены. Приложение корректно отображается на всех устройствах, адаптивная вёрстка перестраивает кнопки оценки в зависимости от ширины экрана. Навигация между страницами работает без ошибок, карточка слова правильно переключает режим вопроса и ответа.

4.3 Модульное тестирование (Unit – тесты)

Для обеспечения качества кода и проверки корректности работы ключевых компонентов были разработаны автоматические модульные тесты с использованием фреймворка Vitest. Тесты охватывают как серверную, так и клиентскую части приложения. В таблице 4.5 представлены результаты выполнения всех модульных тестов.

Таблица 4.5

Результаты модульного тестирования

Тестовый модуль	Файл тестов	Количество	Успешно
-----------------	-------------	------------	---------

		тестов	пройдено
Бэкенд			
Аутентификация	test/auth.test.js	8	8
Слова	test/words.test.js	6	6
Интервальное повторение	test/intervals.test.js	10	10
Администрирование	test/admin.test.js	3	3
Фронтенд			
RatingButtons	tests/RatingButtons.test.jsx	2	2

Продолжение табл. 4.5

StatsBar	tests/StatsBar.test.jsx	1	1
Flashcard	tests/Flashcard.test.jsx	3	3
Итого		33	33

Тестирование серверной части подтвердило корректную работу API аутентификации (регистрация, вход, валидация, защита маршрутов), операций со словами (добавление, оценка, категории) и алгоритма интервального повторения (изменение уровней, граничные значения, полный цикл 0→5, доступность слов по дате повторения). Тестирование клиентской части подтвердило правильный рендеринг React-компонентов (RatingButtons, StatsBar, Flashcard) и корректную обработку событий (нажатие кнопок, клик по карточке). Все 33 теста успешно пройдены, что подтверждает стабильность и надёжность разработанного программного обеспечения.

Наиболее важным для данного проекта является тестирование алгоритма интервального повторения. В листинге 4.1 представлены тесты, проверяющие корректность изменения уровней, граничные значения и полный цикл изучения.

Листинг 4.1 – Тесты алгоритма интервального повторения (server/test/intervals.test.js, фрагмент)

```

describe('Изменение уровней', () => {
  it('Оценка "Отлично" (5) на level 0 → level 1, интервал 1 день', () => {
    {
      console.log('\n--- Тест 1: Отлично на новом слове ---');
      const result = reviewWord(1, 5);
      expect(result.newLevel).toBe(1);
      expect(result.nextReviewDays).toBe(1);
      console.log('Уровень повышен, интервал = 1 день\n');
    }
  });

  it('Оценка "Сложно" (0) понижает уровень', () => {
    console.log('\n--- Тест 3: Сложно понижает уровень ---');
    db.prepare('UPDATE words SET level = 2 WHERE id =
2').run();
    const result = reviewWord(2, 0);
    expect(result.newLevel).toBe(1);
    expect(result.nextReviewDays).toBe(1);
    console.log('Уровень понижен с 2 до 1\n');
  });

  it('Уровень не может быть меньше 0', () => {
    console.log('\n--- Тест 4: Минимальный уровень ---');
    db.prepare('UPDATE words SET level = 0 WHERE id =
3').run();
    const result = reviewWord(3, 0);
    expect(result.newLevel).toBe(0);
    expect(result.nextReviewDays).toBe(0);
    console.log('Уровень остался 0 (не ушёл в минус)\n');
  });

  it('5 отличных оценок подряд: 0 → 1 → 2 → 3 → 4 → 5', () => {
    console.log('\n--- Тест 7: Полный цикл до уровня 5 ---');
    db.prepare("INSERT INTO words (id, user_id, word,
translation, level, next_review) VALUES (4, 1, 'Dog', 'Собака', 0,
datetime('now'))").run();
    console.log('Создано слово "Dog" с level=0');

    const levels = [];
    const intervals = [];
    for (let i = 0; i < 5; i++) {
      const result = reviewWord(4, 5);
      levels.push(result.newLevel);
      intervals.push(result.nextReviewDays);
    }
    console.log(`Уровни: ${levels.join(' → ')}\`);
    console.log(`Интервалы: ${intervals.join(' → ')} дней\`);

    expect(levels).toEqual([1, 2, 3, 4, 5]);
    expect(intervals).toEqual([1, 3, 7, 14, 30]);
    console.log('Полный цикл пройден!\n');
  });
});

```

Результат выполнения тестов алгоритма:

```

Test Files  1 passed (1)
  Tests     10 passed (10)
Start at    17:09:50

```



```
Duration 600ms (transform 47ms, setup 0ms, import 131ms, tests
223ms, environment 0ms)
```

Для проверки надёжности системы аутентификации были разработаны тесты, охватывающие регистрацию, вход, валидацию полей и защиту маршрутов. В листинге 4.2 представлены ключевые тесты API аутентификации.

Листинг 4.2 – Тесты API аутентификации (server/test/auth.test.js, фрагмент)

```
it('Регистрация нового пользователя', async () => {
  console.log('\n--- Тест: Регистрация ---');
  const res = await
request(app).post('/api/auth/register').send({
  username: 'test', email: 'test@test.com', password:
'123456'
  });
  console.log('Статус:', res.status);
  console.log('Ответ:', JSON.stringify(res.body).substring(0,
80) + '...');
  expect(res.status).toBe(201);
  expect(res.body).toHaveProperty('token');
  token = res.body.token;
});

it('Регистрация с коротким паролем', async () => {
  console.log('\n--- Тест: Короткий пароль ---');
  const res = await
request(app).post('/api/auth/register').send({
  username: 'user', email: 'user@test.com', password: '123'
  });
  console.log('Статус:', res.status, '(ожидалось 400)');
  expect(res.status).toBe(400);
});

it('Вход с правильными данными', async () => {
  console.log('\n--- Тест: Успешный вход ---');
  const res = await request(app).post('/api/auth/login').send({
    email: 'test@test.com', password: '123456'
  });
  console.log('Статус:', res.status);
  expect(res.status).toBe(200);
  expect(res.body).toHaveProperty('token');
});

it('Получение профиля без токена', async () => {
  console.log('\n--- Тест: Профиль без токена ---');
  const res = await request(app).get('/api/auth/profile');
  console.log('Статус:', res.status, '(ожидалось 401)');
  expect(res.status).toBe(401);
});
```

```
    });  
  });
```

Тесты проверяют успешную регистрацию нового пользователя, при которой сервер возвращает статус 201 и JWT-токен. При попытке регистрации с паролем длиной менее 6 символов сервер возвращает ошибку 400, что подтверждает корректную работу валидации. Успешный вход с правильными email и паролем возвращает статус 200 и токен. При попытке доступа к защищённому маршруту профиля без токена авторизации сервер возвращает ошибку 401, что подтверждает надёжную защиту маршрутов. Дополнительно тестируются регистрация с пустыми полями (возвращает 400), регистрация с уже занятым email (возвращает 400), вход с неверным паролем (возвращает 401) и получение профиля с действующим токеном (возвращает 200 и данные пользователя). Все тесты успешно пройдены, что подтверждает корректную работу системы аутентификации.

Для проверки корректности работы клиентской части были разработаны тесты для основных React-компонентов с использованием библиотеки `@testing-library/react`. В листинге 4.3 представлен тест компонента кнопок оценки.

Листинг 4.3 – Тест компонента `RatingButtons`
(`client/tests/RatingButtons.test.jsx`, фрагмент)

```
import { render, screen, fireEvent, cleanup } from '@testing-  
library/react';  
import RatingButtons from '../src/components/RatingButtons';  
  
it('отображает кнопки', () => {  
  render(<RatingButtons onRate={() => { }} />);  
  expect(screen.getByText('Сложно')).toBeTruthy();  
  expect(screen.getByText('Отлично')).toBeTruthy();  
});  
  
it('вызывает onRate с 5', () => {  
  const fn = vi.fn();  
  render(<RatingButtons onRate={fn} />);  
  fireEvent.click(screen.getByText('Отлично'));  
  expect(fn).toHaveBeenCalledTimes(5);  
});
```

Первый тест проверяет, что компонент корректно отображает все четыре кнопки оценки. Функция `render` помещает компонент в виртуальное DOM-дерево, а `screen.getByText` находит элементы по их текстовому содержимому. Если хотя бы одна кнопка отсутствует, тест завершится ошибкой.

Второй тест проверяет, что при нажатии на кнопку «Отлично» callback-функция `onRate` вызывается с правильным значением `quality = 5`. Для этого создаётся mock-функция с помощью `vi.fn()`, которая отслеживает вызовы и переданные аргументы. С помощью `fireEvent.click` эмулируется клик пользователя по кнопке, после чего проверяется, что mock-функция была вызвана ровно один раз с аргументом 5. Аналогичные тесты можно добавить для остальных кнопок («Сложно» с `quality=0`, «Не очень» с `quality=2`, «Хорошо» с `quality=4`). Такое тестирование гарантирует, что компонент правильно взаимодействует с родительскими компонентами и передаёт корректные данные в обработчик.

В листинге 4.4 представлен тест компонента карточки слова.

Листинг 4.4 – Тест компонента Flashcard (`client/tests/Flashcard.test.jsx`, фрагмент)

```
import { render, screen, fireEvent, cleanup } from '@testing-
library/react';
import Flashcard from '../src/components/Flashcard';

const word = { id: 1, word: 'Hello', translation: 'Привет', example:
'Hi!', example_translation: 'Привет!', category: 'basics', level: 2 };

it('показывает слово', () => {
  render(<Flashcard word={word} showAnswer={false} onClick={()
=> { }} />);
  expect(screen.getByText('Hello')).toBeTruthy();
});

it('показывает перевод', () => {
  render(<Flashcard word={word} showAnswer={true} onClick={() =>
{ }} />);
  expect(screen.getByText('Привет')).toBeTruthy();
});

it('вызывает onClick', () => {
  let clicked = false;
  render(<Flashcard word={word} showAnswer={false} onClick={()
=> clicked = true} />);
```

```

        fireEvent.click(screen.getByText('Hello'));
        expect(clicked).toBe(true);
    });
});

```

Первый тест проверяет, что в режиме вопроса (когда ответ скрыт) компонент отображает слово на иностранном языке. Второй тест проверяет, что в режиме ответа компонент отображает перевод слова. Оба теста используют `screen.getByText` для поиска элемента с определённым текстом — если элемент не найден, тест завершается ошибкой.

Третий тест проверяет, что при клике на карточку вызывается переданная через пропсы функция `onClick`. Для этого создаётся mock-функция с помощью `vi.fn()`, которая отслеживает вызовы. С помощью `fireEvent.click` эмулируется клик пользователя по карточке, после чего проверяется, что mock-функция была вызвана ровно один раз. Это гарантирует, что родительский компонент сможет переключить режим отображения (вопрос/ответ) при нажатии на карточку. Дополнительно можно протестировать отображение примера использования и перевода примера, когда они переданы в пропсах и включён режим ответа.

Результат выполнения фронтенд – тестов:

```

tests/StatsBar.test.jsx (1 test) 38ms
tests/Flashcard.test.jsx (3 tests) 44ms
tests/RatingButtons.test.jsx (2 tests) 49ms

Test Files   3 passed (3)
Tests       6 passed (6)
Start at    18:17:08
Duration    2.88s (transform 211ms, setup 0ms, import 1.54s,
tests 131ms, environment 6.21s)

```

Разработанные модульные тесты подтверждают корректную работу:

1. Алгоритма интервального повторения – правильное изменение уровней, граничные значения (0 и 5), полный цикл от 0 до 5, доступность слов в зависимости от даты повторения.
2. API аутентификации – регистрация, вход, валидация полей, защита маршрутов.

3. API работы со словами – добавление, оценка, получение категорий, обработка ошибок.

4. React – компонентов – рендеринг, обработка событий, корректное отображение данных.

Все 33 теста (27 бэкенд + 6 фронтенд) успешно пройдены, что подтверждает стабильность и надёжность разработанного программного обеспечения.

4.4 Результат тестирования алгоритма интервального повторения

В ходе тестирования была проверена корректность работы алгоритма интервального повторения. В таблице 4.6 представлен пример работы алгоритма для одного слова с течение нескольких повторений.

Таблица 4.6

Пример работы алгоритма интервального повторения

Действие	Оценка	quality	Уровень до	Уровень после	Интервал
Изучение нового слова	—	—	—	0	0 дней
Первое повторение	Отлично	5	0	1	1 день
Второе повторение	Хорошо	4	1	2	3 дня
Третье повторение	Не очень	2	2	2	3 дня
Четвёртое повторение	Отлично	5	2	3	7 дней
Пятое повторение	Отлично	5	3	4	14 дней
Шестое повторение	Сложно	0	4	3	7 дней

Вывод по результатам тестирования алгоритма:

Алгоритм корректно повышает уровень слова на 1 при успешных ответах (оценки 4 – «Хорошо» и 5 – «Отлично»), оставляет на том же уровне при нейтральной оценке (2 – «Не очень») и понижает на 1 при

ошибке (оценка 0 – «Сложно»), но не ниже минимального уровня 0. Максимальный уровень ограничен 5, что предотвращает выход за допустимые пределы. Интервалы соответствуют фиксированному массиву [0, 1, 3, 7, 14, 30] дней.

Заключение

В ходе выполнения выпускной квалификационной работы была достигнута поставленная цель: разработано и реализовано с использованием JavaScript полнофункциональное веб-приложение для изучения иностранных языков с системой интервального повторения, обеспечивающее эффективное запоминание лексики и отслеживание прогресса пользователя.

Результаты выполненной работы

В соответствии с поставленными задачами были получены следующие результаты:

1. Проведён анализ предметной области. Рассмотрены существующие аналоги (Anki, Quizlet, Duolingo), выявлены их достоинства и недостатки. Изучены алгоритмы интервального повторения SM-2, SM-4 и SM-8. Обоснован выбор упрощённой версии SM-2 как наиболее подходящей для реализации в дипломном проекте.

2. Сформулированы требования к системе. Разработаны функциональные (FR01 – FR07) нефункциональные (NFR01 – NFR05) требования, которые легли в основу проектирования и разработки.

3. Спроектирована архитектура приложения. Разработана клиент-серверная архитектура с REST API. Спроектирована база данных SQLite, включающая таблицы user, words и grammar_rules. Создан составной индекс idx_words_review для оптимизации запросов.

4. Обоснован выбор технологического стека. Для бэкенда выбраны Node.js и Express.js, для базы данных – SQLite с библиотекой better-sqlite3, для фронтенда – React и Vite. Аутентификация реализована с использованием JWT, автоматический перевод – через MyMemory API.

5. Разработана серверная часть. Реализованы маршруты для аутентификации, работы со словами и грамматики. Создано middleware для проверки JWT-токенов. Настроена база данных с использованием прагм WAL, foreign_keys и busy_timeout.

6. Разработана клиентская часть. Созданы React – компоненты: StudyPage (главная страница изучения), Flashcard (карточка слов), RatingButtons (кнопки оценки), StatsBar (панель статистики), AddWordPage (добавление слов), ProfilePage (профиль пользователя), GrammarPage (грамматические правила). Реализована адаптивная вёрстка с тёмной темой оформления.

7. Внедрён алгоритм интервального повторения. Реализована упрощённая версия SM-2 с 6 уровнями изученности (0-5) и фиксированными интервалами [0,1, 3, 7, 14, 30] дней. Логика алгоритма: оценка 4-5 повышает уровень на 1, оценка 2 оставляет без изменений, оценка 0 понижает уровень на 1.

8. Создан стартовый контент. База данных наполнена 30 стартовыми словами по 5 тематическим категориям: «Основы», «Еда», «Путешествия», «Путешествия», «Работа», «Хобби».

9. Проведено тестирование. Выполнено функциональное тестирование (36 сценариев) и модульное тестирование (33

автоматических теста – 27 бэкенд и 6 фронтенд). все тесты успешно пройдены, что подтверждает корректность работы системы.

10. Выполнено сравнение с аналогами. Разработанное приложение не уступает популярным решениям по ключевым критериям, а по некоторым (автоматический перевод, тёмная тема, открытый исходный код) превосходит их.

Практическая значимость работы

Разработанное веб-приложение может быть использовано для самостоятельного изучения иностранной лексики. Ключевые преимущества системы:

- Интуитивно понятный интерфейс – 4 кнопки оценки, тёмная тема, адаптивная вёрстка
- Автоматический перевод – упрощает добавление новых слов
- Открытый исходный код – возможность самостоятельного развёртывания
- Подтверждённая надёжность – 33 автоматических теста подтверждают корректность работы

Направление для дальнейшего развития

В ходе выполнения дипломной работы были выявлены направления для дальнейшего совершенствования приложения:

- Мобильная версия. Разработка мобильного приложения на React Native или прогрессивного веб-приложения (PWA) с возможностью установки на устройство
- Озвучка слов. Интеграция Text-to-Speech API для произношения слов на иностранном языке, что улучшит восприятие на слух
- Облачное хранение. Синхронизация словаря пользователя между несколькими устройствами через облачный сервер

- Поддержка множественных языков. Расширение функционала для изучения не только английского, но и других языков (немецкий, французский, испанский)
- Импорт/экспорт слов. Поддержка импорта и экспорта коллекция слов в формах CSV, Anki (.apkg) для переноса существующих словарей
- Расширенная статистика. Визуализация прогресса в виде графиков и диаграмм, отслеживание «кривой забывания» для каждого слова
- Социальные функции. Возможность делиться колодами слов с друзьями, соревнования по количеству выученных слов

Вывод

Разработанное веб-приложение «Интерактивный тренажёр лексики» представляет собой полноценный программный продукт, готовый к использованию. Оно успешно решает поставленную задачу — обеспечение эффективного запоминания иностранной лексики с использованием алгоритма интервального повторения. Дальнейшее развитие системы позволит расширить её функциональность и аудиторию пользователей.

Список использованных источников

1. SuperMemo. SM-2 Algorithm [Electronic resource]. – URL: <https://www.supermemo.com> (дата обращения: 15.05.2026).
2. React. The library for web and native user interfaces [Electronic resource] / Meta Platforms, Inc. – 2013. – URL: <https://react.dev> (дата обращения: 10.04.2026).
3. Vite. Next Generation Frontend Tooling [Electronic resource]. – URL: <https://vitejs.dev> (дата обращения: 10.04.2026).
4. Express.js. Fast, unopinionated, minimalist web framework for Node.js [Electronic resource]. – URL: <https://expressjs.com> (дата обращения: 10.04.2026).
5. SQLite. Small. Fast. Reliable. Choose any of three [Electronic resource]. – URL: <https://www.sqlite.org> (дата обращения: 10.04.2026).
6. JSON Web Tokens (JWT). RFC 7519 [Electronic resource]. – URL: <https://jwt.io> (дата обращения: 10.04.2026).
7. Better-SQLite3. The fastest and simplest library for SQLite3 in Node.js [Electronic resource]. – URL: <https://github.com/WiseLibs/better-sqlite3> (дата обращения: 10.04.2026).

8. MyMemory API. The world's largest Translation Memory [Electronic resource]. – URL: <https://mymemory.translated.net> (дата обращения: 10.04.2026).
9. MDN Web Docs. Express/Node introduction [Electronic resource] / Mozilla Corporation. – URL: <https://developer.mozilla.org> (дата обращения: 10.04.2026).
10. GitHub. Proprietary developer platform [Electronic resource]. – 2006. – URL: <https://github.com/> (дата обращения: 15.05.2026).

Приложение А

Скриншоты интерфейса веб-приложения

А.1. Страница регистрации

На рис. А.1 представлена форма регистрации нового пользователя. Пользователь вводит имя (не менее 3 символов), email и пароль (не менее 6 символов). После успешной регистрации автоматически выполняется вход в систему и создаётся стартовый набор из 30 слов.

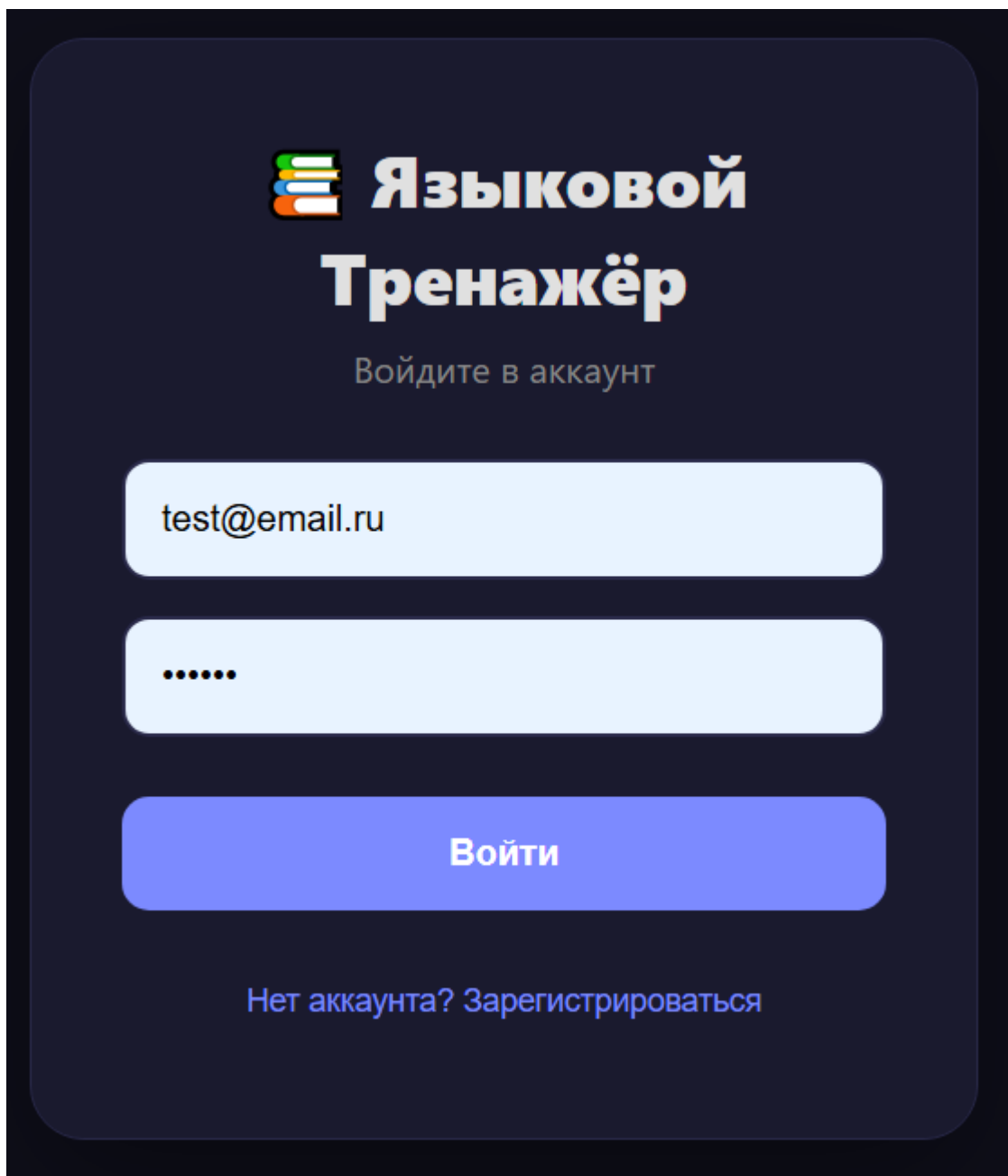


Рис. А.1 – Страница регистрации

Продолжение приложения А

А.2. Главная страница изучения (режим вопроса)

На рис. А.2 представлена главная страница приложения после авторизации. В верхней части отображается панель статистики (общее количество слов, изучено, к повторению) и кнопки выбора количества слов за раз (5, 10, 15, 20). В центре экрана расположена карточка со словом на

иностранном языке. Пользователь должен вспомнить перевод и нажать на карточку.

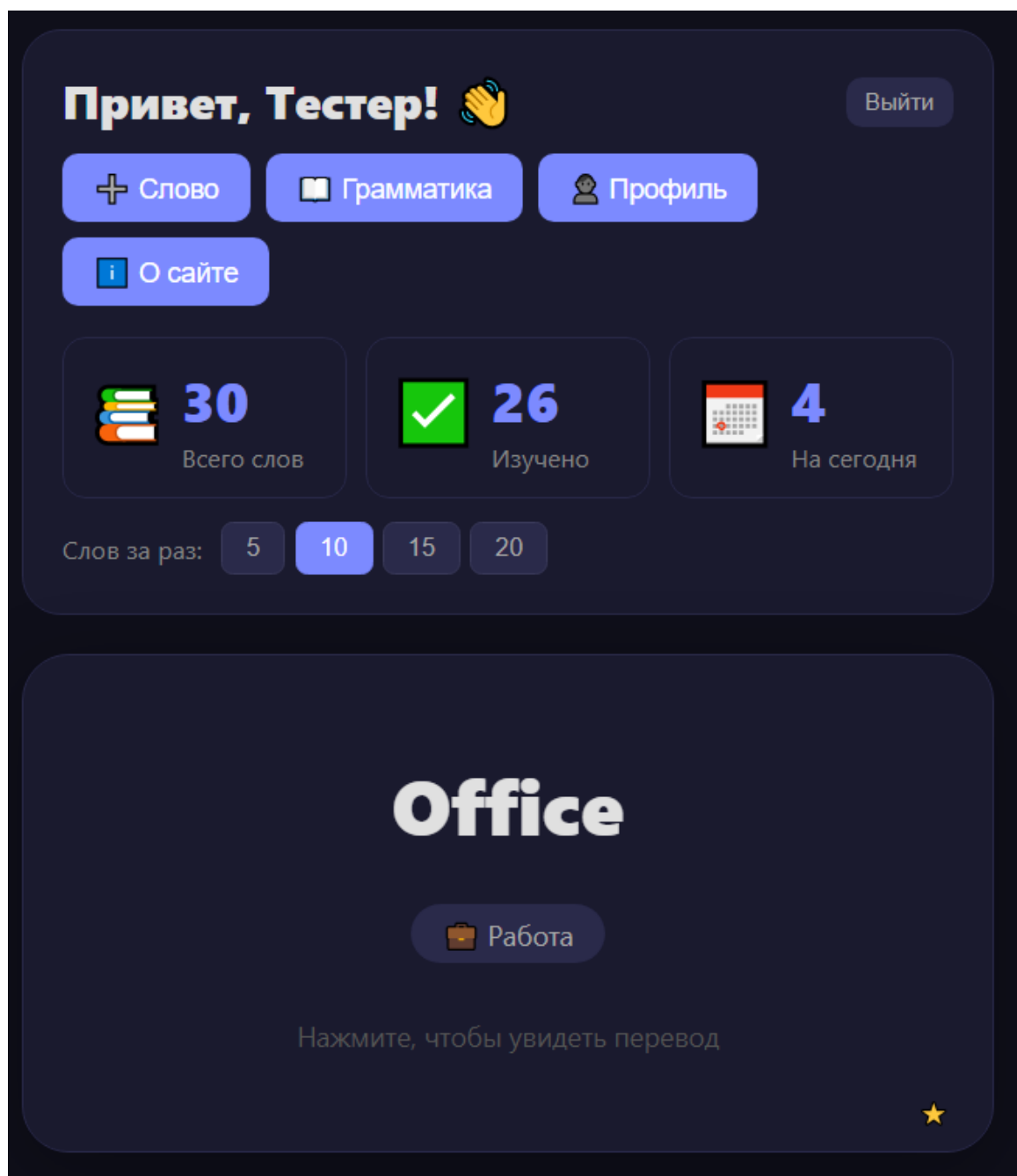


Рис. А.2 – Главная страница изучения (режим вопроса)

Продолжение приложения А

А.3. Карточка слов с ответом

На рисунке А.4 показан вид карточки после нажатия. Отображается перевод слова, пример использования на иностранном языке и его перевод на русский, а также категория слова (с эмодзи). Под карточкой появляются четыре кнопки для оценки качества запоминания.



Рис. А.3 – Карточка слова с ответом

Приложение Б

Листинг программного кода

Листинг Б – Алгоритм интервального повторения и CRUD операции

```
import { Router } from 'express';
import db from '../config/database.js';
import { auth } from '../middleware/auth.js';

const router = Router();
```

```

// Все маршруты защищены
router.use(auth);

// GET /api/words/study - получить слова для изучения
router.get('/study', (req, res) => {
  try {
    const { language = 'en', limit = 10 } = req.query;

    const words = db.prepare(`
      SELECT id, word, translation, example, example_translation,
      category, level, review_count
    FROM words
    WHERE user_id = ?
      AND language = ?
      AND datetime(next_review) <= datetime('now', '+1 day')
    ORDER BY level ASC, RANDOM()
    LIMIT ?
  `).all(req.userId, language, parseInt(limit));

    const stats = db.prepare(`
      SELECT
        COUNT(*) as total,
        SUM(CASE WHEN level >= 3 THEN 1 ELSE 0 END) as learned,
        SUM(CASE WHEN datetime(next_review) <= datetime('now', '+1
day') THEN 1 ELSE 0 END) as due
    FROM words
    WHERE user_id = ? AND language = ?
  `).get(req.userId, language);

    res.json({
      words,
      stats: stats || { total: 0, learned: 0, due: 0 }
    });

  } catch (error) {
    console.error('Get words error:', error);
    res.status(500).json({ error: 'Ошибка получения слов' });
  }
});

// PUT /api/words/:id/review - оценить слово
router.put('/:id/review', (req, res) => {
  try {
    const { id } = req.params;
    const { quality } = req.body; // 0-5

    if (quality === undefined || quality < 0 || quality > 5) {
      return res.status(400).json({ error: 'Оценка должна быть от 0 до 5'
});
    }

    // Проверяем, что слово принадлежит пользователю
    const word = db.prepare(
      'SELECT id, level FROM words WHERE id = ? AND user_id = ?'
    ).get(id, req.userId);
  }
});

```

Продолжение приложения Б

```

        if (!word) {
            return res.status(404).json({ error: 'Слово не найдено'
});
        }

        // Алгоритм интервального повторения
        let newLevel = word.level;
        if (quality >= 4) {
            newLevel = Math.min(5, word.level + 1); // Повышаем
        } else if (quality >= 2) {
            newLevel = word.level; // Оставляем
        } else {
            newLevel = Math.max(0, word.level - 1); // Понижаем
        }

        // Интервалы в днях
        const intervals = [0, 1, 3, 7, 14, 30];
        const daysUntilReview = intervals[newLevel];

        db.prepare(`
UPDATE words
SET level = ?,
    next_review = date('now', '+' || ? || ' days'),
    review_count = review_count + 1
WHERE id = ? AND user_id = ?
`).run(newLevel, daysUntilReview, id, req.userId);

        res.json({
            success: true,
            new_level: newLevel,
            next_review_days: daysUntilReview,
            message: quality >= 4 ? 'Отлично!' : quality >= 2 ?
'Хорошо' : 'Повторим позже'
        });

        } catch (error) {
            console.error('Review error:', error);
            res.status(500).json({ error: 'Ошибка обновления прогресса'
});
        }
    });

// POST /api/words - добавить новое слово
router.post('/', (req, res) => {
    try {
        const { language = 'en', word, translation, example = '',
example_translation = '', category = 'basics' } = req.body;

```

Продолжение приложения Б

```

if (!word || !translation) {
    return res.status(400).json({ error: 'Слово и перевод
обязательны' });
}

// Проверка на дубликат
const existing = db.prepare(

```



```

        'SELECT id FROM words WHERE user_id = ? AND word = ? AND
language = ?'
    ).get(req.userId, word.toLowerCase(), language);

    if (existing) {
        return res.status(400).json({ error: 'Такое слово уже
существует' });
    }

    db.prepare(`
        INSERT INTO words (user_id, language, word, translation, example,
example_translation, category)
        VALUES (?, ?, ?, ?, ?, ?, ?)
    `).run(req.userId, language, word.toLowerCase(),
translation.toLowerCase(), example, example_translation || '',
category);
    res.status(201).json({ message: 'Слово успешно добавлено' });

    } catch (error) {
        console.error('Add word error:', error);
        res.status(500).json({ error: 'Ошибка добавления слова' });
    }
});
// GET /api/words/categories - получить категории
router.get('/categories', (req, res) => {
    try {
        const { language = 'en' } = req.query;

        const categories = db.prepare(`
        SELECT
            category,
            COUNT(*) as total,
            SUM(CASE WHEN level >= 3 THEN 1 ELSE 0 END) as learned
        FROM words
        WHERE user_id = ? AND language = ?
        GROUP BY category
    `).all(req.userId, language);

        res.json({ categories });

    } catch (error) {
        console.error('Categories error:', error);
        res.status(500).json({ error: 'Ошибка получения категорий' });
    }
});
export default router;

```